

Vector Embedded NvM

Technical Reference

Contents

1	Introduction	7
1.1	Architecture Overview	7
1.2	Breaking Changes	7
1.2.1	2.00.00	7
1.2.2	3.00.00	8
2	Functional Description	9
2.1	Features	9
2.1.1	Supported Block Types	9
2.1.1.1	Native	9
2.1.1.2	Redundant	9
2.1.1.3	Dataset	11
2.1.2	Job Processing	11
2.1.2.1	Single Block Job Processing	12
2.1.2.2	MultiBlock Job Processing	13
2.1.3	Monitor Job Status	13
2.1.3.1	Polling	13
2.1.3.2	Single Block Job End Notification	14
2.1.3.3	MultiBlock Job End Notification	15
2.1.3.4	MultiBlock Job Status Information	15
2.1.3.5	Background Crc Recalculation Job End Notification	16
2.1.4	Data Synchronization	16
2.1.4.1	Implicit Synchronization	16
2.1.4.2	Explicit Synchronization	16
2.1.4.3	Temporary Data Buffer	17
2.1.5	Startup (Initialization and NvM_ReadAll)	17
2.1.5.1	Example of a normal startup	18
2.1.5.2	Example of a delayed startup	19
2.1.5.3	Dynamic Configuration	19
2.1.5.4	Skip Reading of Valid Data during NvM_ReadAll()	21
2.1.5.5	Invocation of Callbacks during NvM_ReadAll()	22
2.1.5.6	Kill NvM_ReadAll()	22
2.1.6	Shutdown (NvM_WriteAll and NvM_ValidateAll)	24
2.1.6.1	Cancel NvM_WriteAll()	24
2.1.6.2	Kill NvM_WriteAll()	25
2.1.6.3	Skip Writing of Unchanged Data	25

2.1.7	Automatic Writing of Changed Data (NvM_SetRamBlockStatus and NvM_ValidateAll)	26
2.1.8	Read Data	27
2.1.9	Write Data	27
2.1.10	Invalidate Data	28
2.1.11	Immediate Job Handling	28
2.1.11.1	Single Block Job Interruption	29
2.1.11.2	Multi Block Job Interruption	30
2.1.11.3	Specialties for Multi-partition Projects	33
2.1.11.4	General specialties	34
2.1.12	Erase Data	34
2.1.13	Data Integrity	34
2.1.13.1	Internal Data Integrity Buffer	35
2.1.13.2	Block Size Configuration	35
2.1.13.3	CRC-based Data Integrity	35
2.1.13.4	MAC-based Data Integrity	36
2.1.14	Recovery of Default Data	37
2.1.14.1	Mechanisms for Data Recovery	37
2.1.14.2	Triggers for Data Recovery	38
2.1.15	Write Protection	39
2.1.15.1	Write Block Once	40
2.1.16	Write Retries	41
2.2	Error Handling	41
2.2.1	Development Error Reporting	41
2.2.2	Production Error Reporting	42
2.2.2.1	Multi-partition particularities	42
2.3	Concurrent access to NV data for DCM	42
2.4	Compliance	44
2.4.1	Compliance to previous NvM generation	45
2.4.2	Additions/Extensions	45
2.4.2.1	Support of AUTOSAR 4.4.0 NvMSingleBlockCallback, NvM-MultiBlockCallback and NvMMultiBlockJobStatusInformation	45
2.4.2.2	Support of NVM_REQ_RESTORED_x request result types	45
2.4.2.3	Extended SingleBlock and InitBlockCallbacks	45
3	Integration	46
3.1	Embedded Implementation	46
3.2	Initialization	46
3.3	Main Functions	47

3.4	Synchronization	47
3.4.1	Critical Sections	47
3.5	Memory Sections	47
3.6	Callouts	48
3.7	Error Reporting Det	48
3.8	Block Length Features	48
3.8.1	Automatic Block Length	48
3.8.2	Block Length Check	48
3.8.3	Block Length Features for Explicit Synchronization Blocks	49
3.9	Multi-Partition Projects	51
3.9.1	Multi-Partition communication	51
3.9.2	Multi-Partition Projects and Out-Of-Order Instruction Execution	51
3.9.3	Configurational Particularities	52
3.9.4	Integrational Particularities	52
3.9.4.1	Partitioning	52
3.9.4.2	Startup / Shutdown of MP projects (ReadAll, WriteAll)	53
3.10	Projects without OS Application	53
3.10.1	Configurational Particularities	53
4	API Description	54
4.1	Type Definitions	54
4.2	Provided Functions	54
4.2.1	NvM_CancelJobs	55
4.2.2	NvM_CancelWriteAll	56
4.2.3	NvM_EraseNvBlock	57
4.2.4	NvM_GetActiveMultiBlockApplicationId	58
4.2.5	NvM_GetDataIndex	59
4.2.6	NvM_GetErrorStatus	60
4.2.7	NvM_GetVersionInfo	61
4.2.8	NvM_Init	62
4.2.9	NvM_InvalidateNvBlock	63
4.2.10	NvM_KillReadAll	64
4.2.11	NvM_KillWriteAll	65
4.2.12	NvM_MainFunction	66
4.2.13	NvM_ReadAll	67
4.2.14	NvM_ValidateAll	68
4.2.15	NvM_ReadBlock	69
4.2.16	NvM_ReadPRAMBlock	70
4.2.17	NvM_RestoreBlockDefaults	71

4.2.18	NvM_RestorePRAMBlockDefaults	72
4.2.19	NvM_SetBlockLockStatus	73
4.2.20	NvM_SetBlockProtection	74
4.2.21	NvM_SetDataIndex	75
4.2.22	NvM_SetRamBlockStatus	76
4.2.23	NvM_WriteAll	77
4.2.24	NvM_WriteBlock	78
4.2.25	NvM_WritePRAMBlock	79
4.3	Required Functions	79
4.4	Service Ports	80
4.4.1	Client Server Interface	80
4.4.1.1	Provided Ports	80
4.4.1.2	Required Ports	81
5	Configuration	82
5.1	Configuration Variants	82
6	Cybersecurity	83
7	Glossary and Abbreviations	84
8	Contact	85

Document Information

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_CP_SWS_NVRAMManager.pdf	R23-11
[2]	Vector	AN-ISC-2-1282_NvMGen2_Integration_Multi-Partition-Projects.pdf	1.00.00



Caution

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

1 Introduction

This document describes the functionality, integration, API and configuration of the component NvM.

The NvM is specified in AUTOSAR [1].

The NvM abstracts the management and maintenance of non-volatile memory. The non-volatile memory access is block based and shall only be managed by the NvM in order to avoid conflicts and inconsistent data.

The module accesses the MEMIF which is abstracting the FEE and EA modules. Therefore, the NvM is hardware independent.



Note

MICROSAR FEE and MICROSAR EA are different products that are not part of MICROSAR NVM!

1.1 Architecture Overview

These are the interfaces of the component:

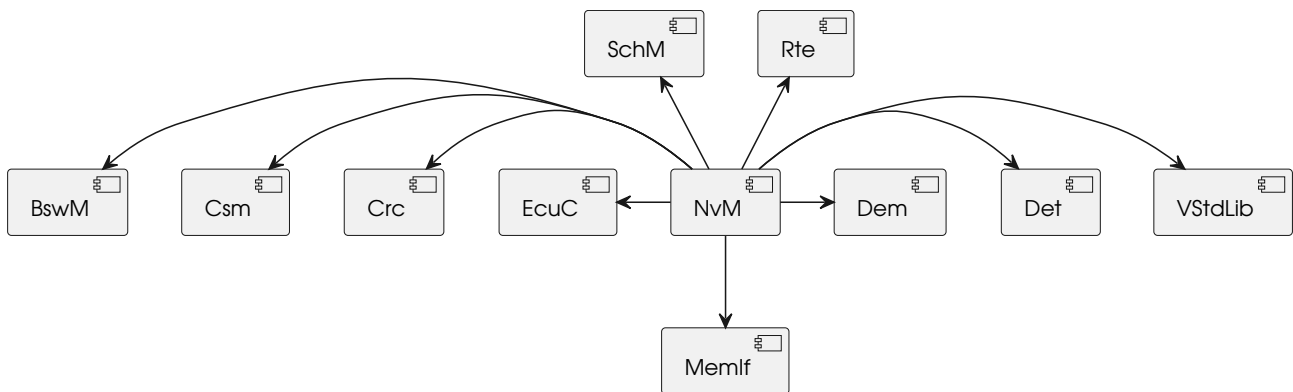


Figure 1.1 Component Architectural Environment

1.2 Breaking Changes

This section provides documentation for breaking changes of the component.

1.2.1 2.00.00

This version improved the behavior of data integrity checks when `NvM/NvMCommon/NvMUseBlockIdCheck` is enabled.

Background

The component provides defines with a unique identifier of each configured block (`NvMConf_NvMBlockDescriptor_NvMBlock_<ShortNameOfBlock>`). This identifier cannot be effectively reused for internal mechanisms such as lookups in management tables; especially in multi-partition projects. Therefore an internal and an external identifier is existing.

The component did use the internal identifier to generate data integrity records. This leads to reasonable and valid checks but was inaccurate. Therefore, this handling was improved to use the truly unique identifier.

Impact

This change leads to loss of data for each block with the following configuration:

- > `NvM/NvMCommon/NvMUseBlockIdCheck` **enabled**
- > `NvM/NvMBlockDescriptor/NvMBlockUseCrc` **enabled**

1.2.2 3.00.00

This version aligns the behavior of the internal CRC mechanism to the one implemented in the legacy NvM for blocks with enabled `NvM/NvMBlockDescriptor/NvMBlockUseCrc`.

Background

Prior to alignment, NvMGen2 consistently handled CRC calculations in Little-Endian format, whereas the legacy NvM adjusted its CRC handling based on the system's endianness. Consequently, migrating from a legacy NvM with a Big-Endian system to NvMGen2 could result in data loss. To prevent this, NvMGen2's CRC handling was aligned with that of the legacy NvM.

Impact

This change can lead to loss of data in Big-Endian systems for each block with the following configuration:

- > `NvM/NvMBlockDescriptor/NvMBlockUseCrc` **enabled**

2 Functional Description



Info

Feature references (*"This feature was initially created considering..."*) shall provide transparency regarding the specification we considered while implementing the feature.

This information is not suitable for checking the compliance of this component against the mentioned standards. For compliance / deviation information, please refer to the subsequent compliance chapter.

2.1 Features

2.1.1 Supported Block Types

The module supports multiple block management types. This can be configured via the parameter `NvM/NvMBlockDescriptor/NvMBlockManagementType`.

This feature was initially created considering AUTOSAR 23-11.

2.1.1.1 Native

Simple block type. The data is stored at one position.

2.1.1.2 Redundant

The data is stored twice.

Reading Redundant Data:

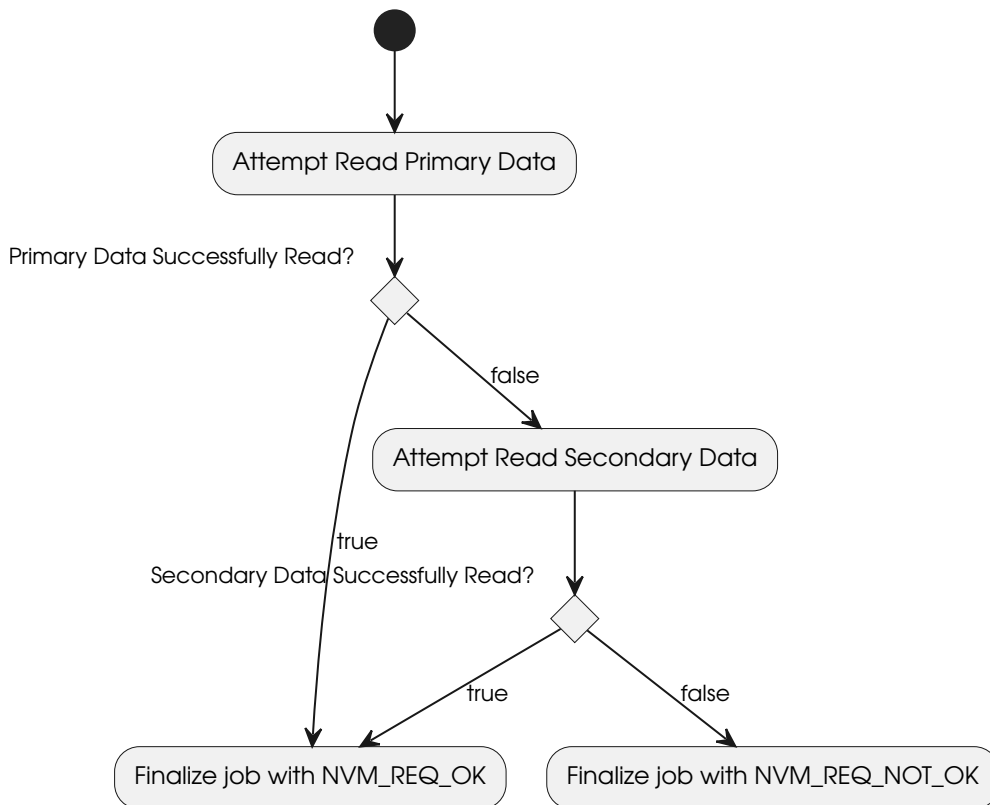


Figure 2.1 Reading logic of a redundant block

Writing Redundant Data:

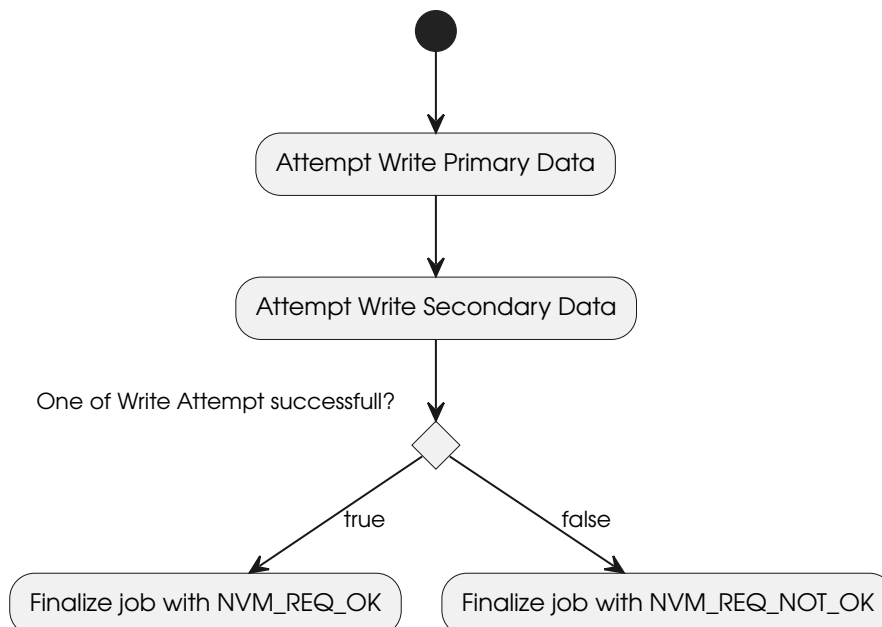


Figure 2.2 Writing logic of a redundant block

Invalidating/Erasing Redundant Data:

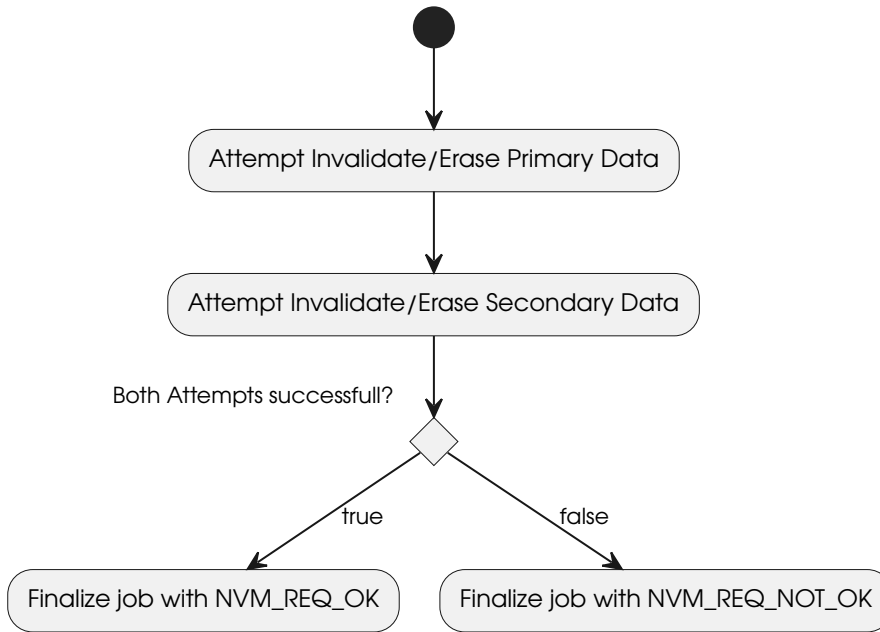


Figure 2.3 Invalidating/Erasing logic of a redundant block

2.1.1.3 Dataset

A set of data.

The amount of possibly addressed data sets can be configured via the parameter `NvM/NvMCommon/NvMDatasetSelectionBits`.

The application has to manage the currently used index of the data set. On startup this index is always automatically set to be zero.

API services to manage the used index:

- > `NvM_GetDataIndex()`
- > `NvM_SetDataIndex()`

2.1.2 Job Processing

Most features of the module are processed asynchronous. That means a service API requests a certain job which will be processed within the main function.

It is documented within Provided Functions if a provided service API is asynchronous or synchronous.

This feature was initially created considering AUTOSAR 23-11.

2.1.2.1 Single Block Job Processing

The module supports queueing of multiple single block requests. The processing order of the queued jobs can be controlled by `NvM/NvMCommon/NvMJobPrioritization`. If enabled the queued jobs are processed based on their priority assigned at `NvM/NvMBlockDescriptor/NvMTargetBlockReference/NvMJobPrioritization`. In case of jobs with equal priority values or all together disabled prioritization the FIFO principle is applied. The size of this queue can be configured via the parameter `NvM/NvMCommon/NvMSizeStandardJobQueue`.



Caution

It is not possible to request multiple operations for one block.

For each configured block an identifier is generated in `NvM_CfgDefines.h`. These identifiers shall be used when using single block service API's.

If the NvM access is done via RTE, this should be abstracted via generated services.

2.1.2.1.1 Cancel single block job

It is possible to cancel queued jobs in a non-destructive manner. Using the `Nvm_CancelJobs()` API, all jobs with associated `BlockId` will be removed from the queue. As a result jobs already in active processing cannot be canceled using this API.

2.1.2.1.2 Multi-partition particularities

The selection of satellite requests follows two different mechanisms:

- > Disabled job prioritization: The master selects the next satellite job request, based on the round-robin principle. The round-robin principle is applied to ensure fair distribution, so that no single satellite is consistently selected for processing.
- > Enabled job prioritization: The master selects the next satellite job request based on its block priority configuration, including the round-robin principle. Round-Robin is necessary for blocks with the same priority, so that no single satellite is consistently selected for processing.

2.1.2.1.3 Shared data across multiple applications

Sometimes block data is important for multiple applications. Nonetheless, from the perspective of the NvM, a block is related to a single application. This is limited by the way the data synchronization is specified by AUTOSAR. More on that is described in the related chapter (Data Synchronization).

For single partition projects it is theoretically possible to have multiple applications requesting data for a single block. But it is the responsibility of the applications to synchronize the data access.

For multi-partition projects it is not sufficient to do so. Shared data must be distributed on the upper layers.



Caution

Generally it is highly recommended to not have multiple users of one block. If data is relevant for multiple applications it shall be distributed at the upper layer.

2.1.2.2 MultiBlock Job Processing

Processing of multiblock jobs should be handled by the BSW Mode Manager.

2.1.3 Monitor Job Status

This chapter describes the supported types of monitoring a job status.

This feature was initially created considering AUTOSAR 23-11.

2.1.3.1 Polling

Each asynchronous job can be observed via the `NvM_GetErrorStatus()` service API.

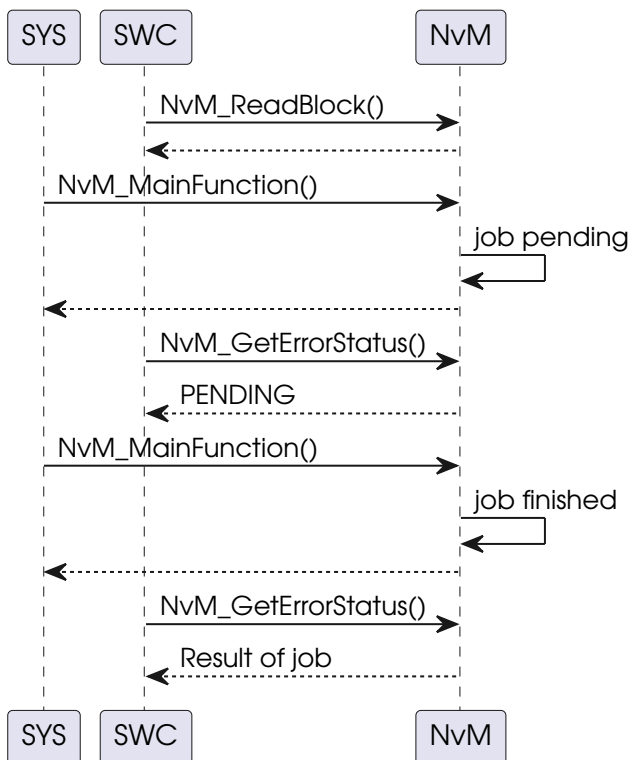


Figure 2.4 Example of a polling scenario

2.1.3.2 Single Block Job End Notification

This callback will be invoked whenever any single block job for a block was finished. Additionally it will be invoked when `NvM_ReadAll()` has processed the block to enable a faster startup for the application of the certain block. For this two options are provided:

- > AUTOSAR conformant `SingleBlockCallback`
- > Non AUTOSAR conformant `SingleBlockCallbackExtended`

The difference is that `SingleBlockCallback` provides `requestID` and `jobResult`. To learn about the reason for the extended callback see `ExtendedSingleBlock` and `InitBlockCallbacks`.



Note

`NvM_WriteAll()` is not invoking this callback as it is expected that the system is going to shutdown and no application is running anymore. This is also a requirement defined by AUTOSAR.



Note

It is possible to configure both `SingleBlockCallback` and `SingleBlockCallbackExtended` at the same time. In this case they will be called in this order. It is up to the integrator to ensure the desired behavior is properly split between both callbacks.

Following parameters have to be configured to enable `SingleBlockCallback`:

- > `NvM/NvMBlockDescriptor/NvMUseJobendCallback=TRUE`
- > `NvM/NvMBlockDescriptor/NvMSingleBlockCallback=<function pointer>`

The following parameter has to be configured to enable `ExtendedSingleBlockCallback`: -
`NvM/NvMBlockDescriptor/NvMSingleBlockCallbackExtended=<function pointer>`



Note

`NvM/NvMBlockDescriptor/NvMUseJobendCallback=TRUE` is not required here as this callback is not AUTOSAR conformant. Simply leaving this parameter empty disables usage of the extended callback.

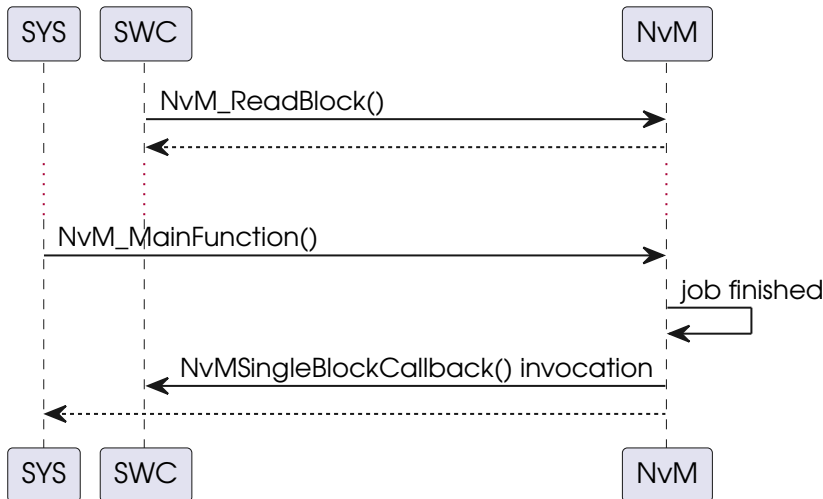


Figure 2.5 Example of a single block job end notification scenario

2.1.3.3 MultiBlock Job End Notification

This callback gets invoked whenever a multiblock job was finished.

To enable this feature, the parameter `NvM/NvMCommon/NvMMultiBlockCallback=<function pointer>` must be configured.

2.1.3.4 MultiBlock Job Status Information

This parameter specifies whether BswM is informed about the current status of the multiblock job.

The MultiBlock Job Status Notification, not only notifies a finished multiblock job, but also a started multiblock job.



Note

MultiBlock Job Status Notification only notifies the start and finish of ReadAll and WriteAll jobs.

To enable this feature set following parameter `NvM/NvMCommon/NvMBswMMultiBlockJobStatusInformation=TRUE`.



Note

Enabling this feature, disables the MultiBlock Job End Notification feature.

2.1.3.5 Background Crc Recalculation Job End Notification

This callback will be invoked whenever any background crc recalculation for a block was finished.

Following parameters have to be configured to enable this feature:

- > NvM/NvMBlockDescriptor/NvMCalcRamBlockCrc=TRUE
- > NvM/NvMBlockDescriptor/NvMBackgroundCrcRecalcCallback=<function pointer>

The callback is defined as stated below:

- > void <BackgroundCrcRecalcCallbackName>(NvM_RequestResultType requestResult)

The request result can be one of two values:

- > NVM_REQ_OK => Finished **successfully**
- > NVM_REQ_NOT_OK => Finished **unsuccessfully**

2.1.4 Data Synchronization

The data synchronization strategy can be configured per block.

This feature was initially created considering AUTOSAR 23-11.

2.1.4.1 Implicit Synchronization

When configuring the parameter NvM/NvMBlockDescriptor/NvMRamBlockDataAddress=<symbol>, the application and the NvM are sharing the configured buffer.

The configured symbol will automatically be used whenever any read or write service is requested for this block.



Caution

The shared buffer must not be altered whenever an asynchronous service for this block is running. Altering the buffer can likely lead to data inconsistencies.



Note

The parameter NvM/NvMBlockDescriptor/NvMBlockUseSyncMechanism must be disabled.

2.1.4.2 Explicit Synchronization

If the parameter NvM/NvMBlockDescriptor/NvMBlockUseSyncMechanism is enabled, the data exchange is done via two separately configurable callbacks:

- > NvM/NvMBlockDescriptor/NvMReadRamBlockFromNvCallback=<symbol>, invoked for reading services
- > NvM/NvMBlockDescriptor/NvMWriteRamBlockToNvCallback=<symbol>, invoked for writing services

The implementation of the configured callbacks must ensure the correct data transfer to and from the applications buffer.

2.1.4.3 Temporary Data Buffer

Some services provide the possibility to use temporary data buffers for data exchange:

- > NvM_ReadBlock()
- > NvM_WriteBlock()
- > NvM_RestoreBlockDefaults()

Whenever a valid pointer (not NULL_PTR) is given to these services, the configured data exchange mechanism will be by-passed and the given source/target pointer will be used.



Caution

The caller of these services must ensure that the targeted buffer has sufficient length for the block data.

NvM does not have the possibility to verify the length and therefore the sufficient data length is essential to avoid out of bound access.

2.1.5 Startup (Initialization and NvM_ReadAll)

As designed by AUTOSAR the initialization of the NvM is completely done after NvM_Init() and NvM_ReadAll() was completely processed.

Nonetheless the MICROSAR NvM is able to process single block jobs before NvM_ReadAll() is processed. When single block jobs are enqueued and NvM_ReadAll() is requested, all queued single block jobs will be processed before starting the multiblock request.

This can make sense for some projects that require data to be read out before the normal initialization phase starts. In this case the integrator has to take care that the service APIs are invoked correctly.



Note

There is no limitation to the type of single block job that can be performed. The correct order of the jobs has to be ensured by the integrator in this case (e.g. reading data before writing).

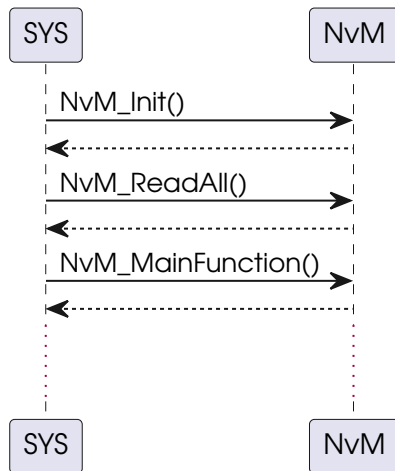


Caution

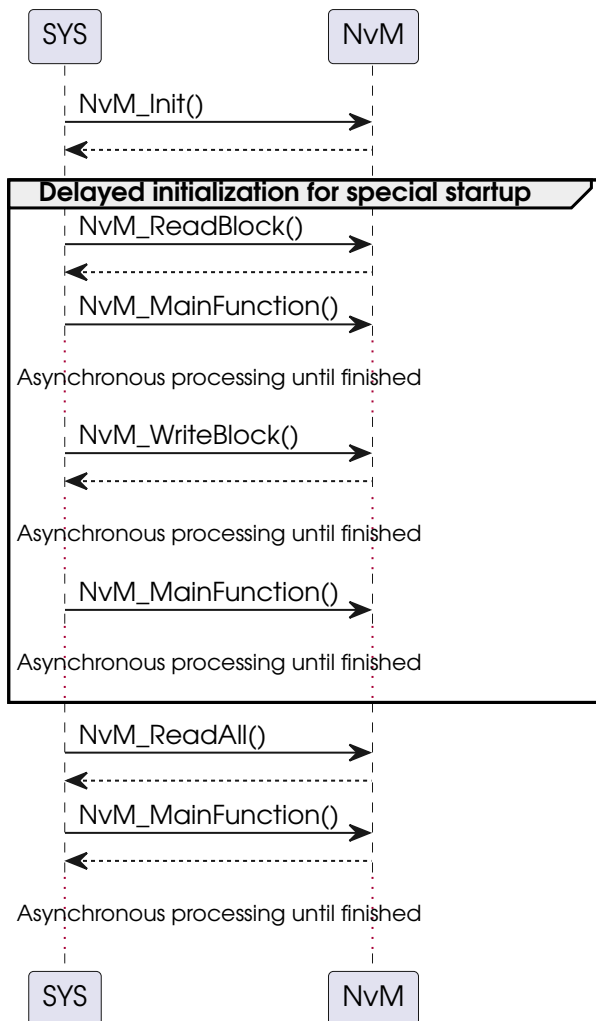
Only blocks with the parameter `NvM/NvMBlockDescriptor/NvMSelectBlockForReadAll=TRUE` will be processed.

This feature was initially created considering AUTOSAR 23-11.

2.1.5.1 Example of a normal startup



2.1.5.2 Example of a delayed startup



2.1.5.3 Dynamic Configuration

The configuration block is a pre-configured redundant NvM block with identifier 1. The block contains the configuration id (also stored in NVRAM), which is used to detect configuration updates during startup (NvM_ReadAll). The NVRAM content of the block is compared to the NvM/NvMCommon/NvMCompiledConfigId/NvMCompiledConfigId or alternatively via Callback see Provide Configuration ID as a callback to recognize an update.



Note

The configuration id is a static value and must be provided by the user either via NvM/NvMCommon/NvMCompiledConfigId/NvMCompiledConfigId or alternatively via Callback see Provide Configuration ID as a callback.

Depending on the result of the block comparison and the configuration parameter NvM/NvMBlockDescriptor/NvMResistantToChangedSw, each block is processed either in a normal or extended runtime preparation.

**Caution**

The configuration block is only generated in case `NvM/NvMCommon/NvMDynamicConfiguration` is enabled to reduce the footprint of the module.

2.1.5.3.1 Normal Runtime Preparation

The following situations lead to a normal runtime preparation:

- > Dynamic Configuration Disabled (configuration block is not relevant at all)
- > Dynamic Configuration Enabled and no configuration id mismatch occurred
- > Dynamic Configuration Enabled and configuration id mismatch occurred but block is resistant to changed SW

Normal runtime preparation means that the block is handled normally by `ReadAll`.

2.1.5.3.2 Extended Runtime Preparation

The following situations lead to a extended runtime preparation:

- > Dynamic Configuration Enabled, block **not** resistant to changed SW and configuration id mismatch occurred
- > Dynamic Configuration Enabled, block **not** resistant to changed SW and configuration block cannot be read

Extended runtime preparation means:

- > Configuration block is updated and prepared to be rewritten during next `WriteAll` request
- > If default data is configured:
 - > The block's default data is restored, and its error status is set to `NVM_REQ_RESTORED_DEFAULTS`.
- > If default data is not configured:
 - > No default data is restored, and the block's error status is set to `NVM_REQ_INTEGRITY_FAILED`.

2.1.5.3.3 Provide Configuration ID as a callback

As an alternative to the `NvMCompiledConfigId`, the configuration parameter `NvM/NvMCommonVendorParams/NvMProvideDynamicConfigIdCallback` can be used.

**Expert Knowledge**

The callback routine must follow the type definition of `NvM_ConfigIdCbKPtrType`.

**Note**

When using the `NvMProvideDynamicConfigIdCallback`, the block length of the configuration block can be set to any valid `NvMNvBlockLength` value. The block length is set to the fixed size of 2 if the `NvMCompiledConfigId` is used.

**Caution**

When changing to `NvMProvideDynamicConfigIdCallback` it is most likely, that a configuration update is executed. The configuration ID as well as a possible change in block length leads to either a mismatch of configuration ID or undiscoverable configuration block.

2.1.5.4 Skip Reading of Valid Data during `NvM_ReadAll()`

The module provides the possibility to skip reading of NV data for a block during `NvM_ReadAll()` processing if the following conditions are met:

- > `NvM/NvMBlockDescriptor/NvMSelectBlockForReadAll` is enabled
- > `NvM/NvMBlockDescriptor/NvMBlockUseCrc` is enabled
- > `NvM/NvMBlockDescriptor/NvMCalcRamBlockCrc` is enabled
- > `NvM/NvMCommonVendorParams/NvMDataIntegrityIntBuffer` is enabled and mapped to a memory section which survives a reset
- > The block has `NvM/NvMBlockDescriptor/NvMBlockUseSetRamBlockStatus` enabled and the ram block status is **VALID** (must also be mapped to a memory section which survives a reset)
- > The CRC record recalculated for Ram Block Data Buffer (Implicit or Explicit Synchronization) matches the stored CRC record in the `NvM/NvMCommonVendorParams/NvMDataIntegrityIntBuffer`

If the NV read is skipped, the error status of the block is `NVM_REQ_OK`.

**Note**

Enabling this feature can lead to faster startup times, as reading from NV is time-consuming.

**Expert Knowledge**

The blocks RAM block status does not change if NV read is skipped for blocks with valid data during `NvM_ReadAll()`

2.1.5.4.1 Background CRC Recalculation

If `NvM/NvMBlockDescriptor/NvMCalcRamBlockCrc` is enabled, calling `NvM_SetRamBlockStatus` API triggers a background recalculation of the CRC. The resulting CRC then gets stored in the `NvM/NvMCommonVendorParams/NvMDataIntegrityIntBuffer`, which later can be used for comparison of the CRC record, as described in Skip Reading of Valid Data during `NvM_ReadAll()`.



Expert Knowledge

In case `NvM_SetRamBlockStatus(<BlockId>, TRUE)` is called, the corresponding block is enqueued and the CRC recalculation is performed within the background.



Caution

The background CRC recalculation mechanism is neither priority based nor based on a FIFO queue implementation. There is mechanism available for the user to determine when the background recalculation for a block is finished. For more information about this mechanism reference Background Crc Recalculation Job End Notification.

2.1.5.5 Invocation of Callbacks during `NvM_ReadAll()`

Some components might not be fully initialized when processing `NvM_ReadAll()` (e.g., during startup). Calling callbacks for these components can result in errors. To address this, the NvM offers the configuration parameter `NvM/NvMBlockDescriptor/NvMInvokeCallbacksForReadAll=<value>`, which allows you to disable the invocation of `NvMSingleBlockCallback` and `NvMInitBlockCallback` for a block during `NvM_ReadAll()`.

2.1.5.6 Kill `NvM_ReadAll()`



Note

This is a MICROSAR feature, extending the AUTOSAR standard.

In special circumstances the execution of `NvM_ReadAll()` can consume too much time to fulfil the targeted startup time. For these scenarios MICROSAR supports the service API `NvM_KillReadAll()`. Triggering this service will immediately abort the running `NvM_ReadAll()` service.

Please note that this service is considered to be the last instance and is a destructive service.

**Caution**

The integration of this service should be done very carefully. Killing the normal startup process can potentially create severe runtime problems for the applications as they normally rely on having valid data when starting up.

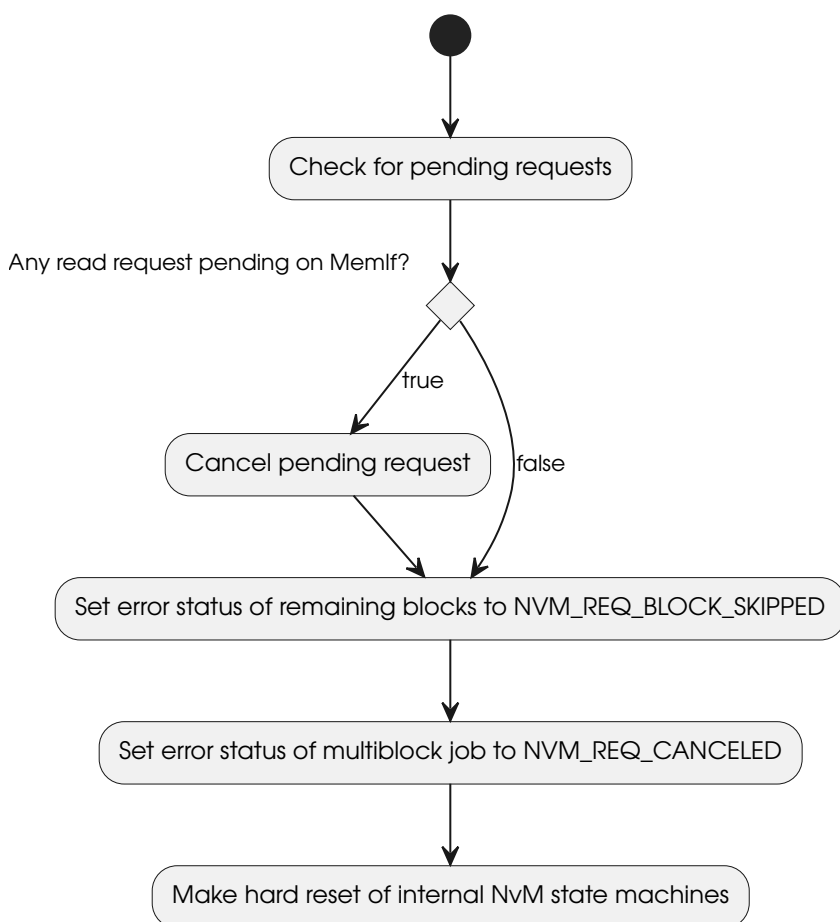
**Caution**

All blocks that were not yet processed by the killed service, have to be manually read via `NvM_ReadBlock()`.

**Caution**

This service is currently only supported for single partition projects.

2.1.5.6.1 Execution Logic



2.1.6 Shutdown (NvM_WriteAll and NvM_ValidateAll)

The shutdown of the ECU requires changed data to be written. This is initiated via the service API NvM_WriteAll().

Before this service is started, all enqueued single block jobs are processed.



Caution

Only blocks that were marked as changed will be written, see Automatic Writing of Changed Data (NvM_SetRamBlockStatus and NvM_ValidateAll) for more details.



Caution

Only blocks with the parameter NvM/NvMBlockDescriptor/NvMSelectBlockForWriteAll=TRUE will be processed.

This feature was initially created considering AUTOSAR 23-11.

2.1.6.1 Cancel NvM_WriteAll()

In special circumstances the execution of NvM_WriteAll() can consume too much time to fulfil the targeted shutdown time. For these scenarios MICROSAR supports the service API NvM_CancelWriteAll().

This service will wait until a potentially pending block request has finished, and will not continue to process any other block. After that, the service will be finalized with the result NVM_REQ_CANCELED.



Caution

The integration of this service should be done very carefully. Updated information of the blocks that could not be processed will be lost if not manually written.



Caution

This service is currently only supported for single partition projects.



Note

If a WriteAll process is successfully canceled, the NvMMultiBlockCallback is called twice. First, the callback is invoked for WriteAll with the error status NVM_REQ_CANCELED, then the callback is called for CancelWriteAll with the same error status.

2.1.6.2 Kill NvM_WriteAll()

In the rare case the shutdown still takes too long to fulfil the targeted time constraint after cancelling the service, the service API NvM_KillWriteAll() can be used to enforce the finalization of the shutdown process.

Please note that this service is considered to be the last instance and is a destructive service.



Caution

This service is currently only supported for single partition projects.

2.1.6.3 Skip Writing of Unchanged Data

The module provides the possibility to skip writing of NV data for a block during NvM_WriteAll() (resp. NvM_WriteBlock(), resp. NvM_WritePRAMBlock()) processing if the following conditions are met:

- > NvM/NvMBlockDescriptor/NvMBlockUseCrc is enabled
- > NvM/NvMBlockDescriptor/NvMBlockUseCRCCompMechanism is enabled
- > Calculated CRC matches the stored CRC record



Expert Knowledge

The CRC record used for comparison is stored within an internal comparison buffer, which is updated after each successful write or read job of the block. Enabling this feature can lead to faster shutdown times, as writing to NV is time-consuming. Nevertheless, this feature also applies for single write block requests.

The following diagram describes the evaluation of the skip writing:

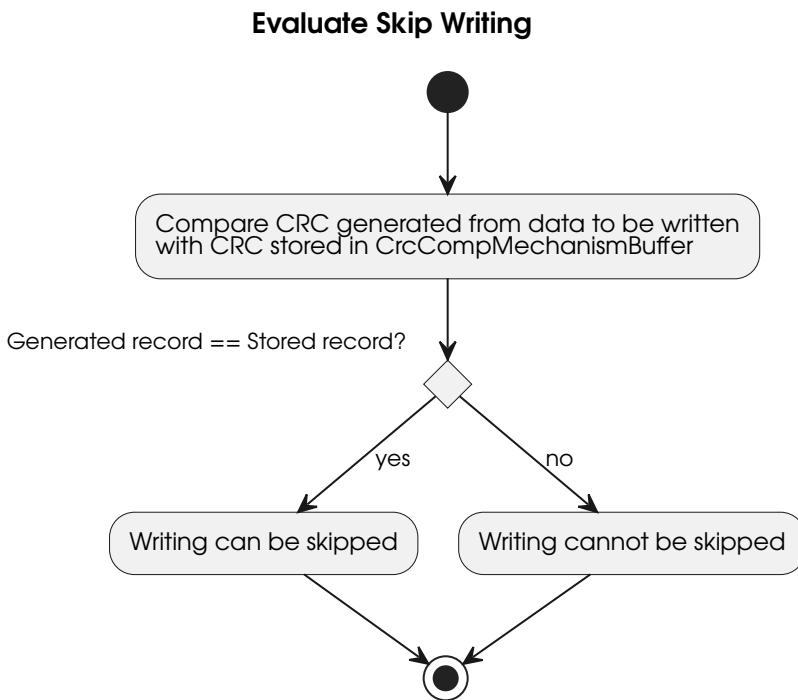


Figure 2.6 Evaluate Skip Writing

This feature was initially created considering AUTOSAR 23-11. These are the major deviations compared to the used AUTOSAR version [1]

> Currently no loss of redundancy check is considered during skip writing of valid data.

2.1.7 Automatic Writing of Changed Data (NvM_SetRamBlockStatus and NvM_ValidateAll)

It is possible to configure blocks that are automatically read and written via the startup and shutdown services. From the perspective of the NvM it is not possible to know which block information has changed. This is where NvM_SetRamBlockStatus() comes in place. This service API must be used to inform the NvM whenever the block data has changed.

Based on the information of the latest NvM_SetRamBlockStatus() invocation, the NvM will decide whether the data will be written during the shutdown or not.

For a aggregated validation of blocks, the NvM_ValidateAll() API can be invoked, which will set the RamBlockState to VALID/CHANGED for all blocks with NvM/NvMBlockDescriptor/NvMBlockUseAutoValidation enabled. In case NvM/NvMBlockDescriptor/NvMCalcRamBlockCrc=TRUE, a background CRC recalculation is triggered additionally.



Caution

Using NvM_SetRamBlockStatus(<blockId>, FALSE) will indicate the NvM that the data has not been changed. Independently if a changed data was indicated before, this will ultimately inform the NvM that the block shall not be written.

**Caution**

In case of a `NvM_ValidateAll()` invocation, only the multiblock job notifications are performed, but no singleblock notifications. Additionally, blocks which are not processed during `NvM_ValidateAll()` are not marked as SKIPPED, but keep the error status as it was before.

**Expert Knowledge**

`NvM_ValidateAll()` shall be invoked before shutdown to get the desired effect of validating multiple blocks at once, which will be written during `NvM_WriteAll()`. To use `NvM_ValidateAll()` in a proper way one have to enable both `NvM/NvMBlockDescriptor/NvMBlockUseAutoValidation` and `NvM/NvMBlockDescriptor/NvMBlockUseSetRamBlockStatus`.

This feature was initially created considering AUTOSAR 23-11.

2.1.8 Read Data

Reading NV data is possible via following service API's:

- > `NvM_ReadBlock()`
- > `NvM_ReadPRAMBlock()`

The PRAM variant of the API can be used to read data and automatically use the configured data exchange mechanism.

The functionality of both API variants is basically the same. The main difference is that `NvM_ReadPRAMBlock()` does not provide the possibility to use a temporary buffer.

**Note**

Using `NvM_ReadBlock(<blockId>, NULL_PTR)` is the equivalent to `NvM_ReadPRAMBlock(<blockId>)`.

This feature was initially created considering AUTOSAR 23-11.

2.1.9 Write Data

Writing NV data is possible via following service API's:

- > `NvM_WriteBlock()`
- > `NvM_WritePRAMBlock()`

The PRAM variant of the API can be used to write data and automatically use the configured data exchange mechanism.

The functionality of both API variants is basically the same. The main difference is that `NvM_WritePRAMBlock()` does not provide the possibility to use a temporary buffer.

**Note**

Using `NvM_WriteBlock(<blockId>, NULL_PTR)` is the equivalent to `NvM_WritePRAMBlock(<blockId>)`.

This feature was initially created considering AUTOSAR 23-11.

2.1.10 Invalidate Data

To make an NV block unreadable, data can be invalidated.

Invalidating NV data is possible via following service API:

> `NvM_InvalidateNvBlock()`

This feature was initially created considering AUTOSAR 23-11.

2.1.11 Immediate Job Handling

An immediate block job is a write block request for a block with priority 0 (configured via `NvM/NvMBlockDescriptor/NvMBlockJobPriority`). Those requests shall be processed as soon as possible, which results in interrupting already ongoing single block jobs (rep. multi block jobs). Such an interruption is initiated by canceling an ongoing job at the level of the underlying MemIf. Immediate block jobs are queued into a separate immediate job queue with a configurable size via the parameter `NvM/NvMCommon/NvMSizeImmediateJobQueue`, which is processed with higher priority than the normal single block job queue, meaning that the immediate queue is always processed first, see:

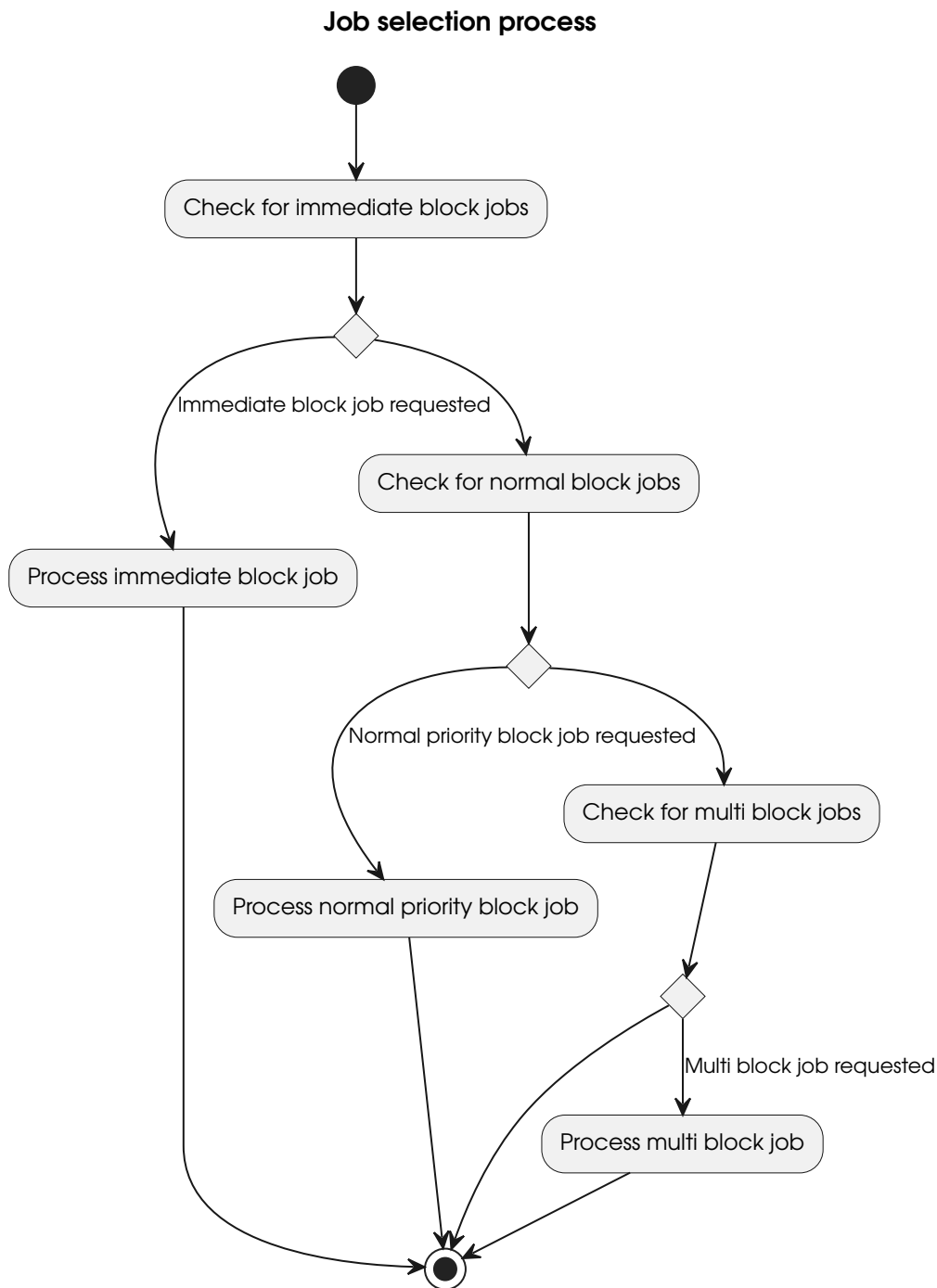


Figure 2.7 Job selection process

2.1.11.1 Single Block Job Interruption

In case a single block job is already ongoing when an immediate block job is requested, the ongoing job will be interrupted and the immediate block job will be started right after the interruption. After the immediate block job is finished, the interrupted job will be restarted from the beginning. The following diagram abstractly describes how the interruption of a single block job is performed in case of an immediate block job request.

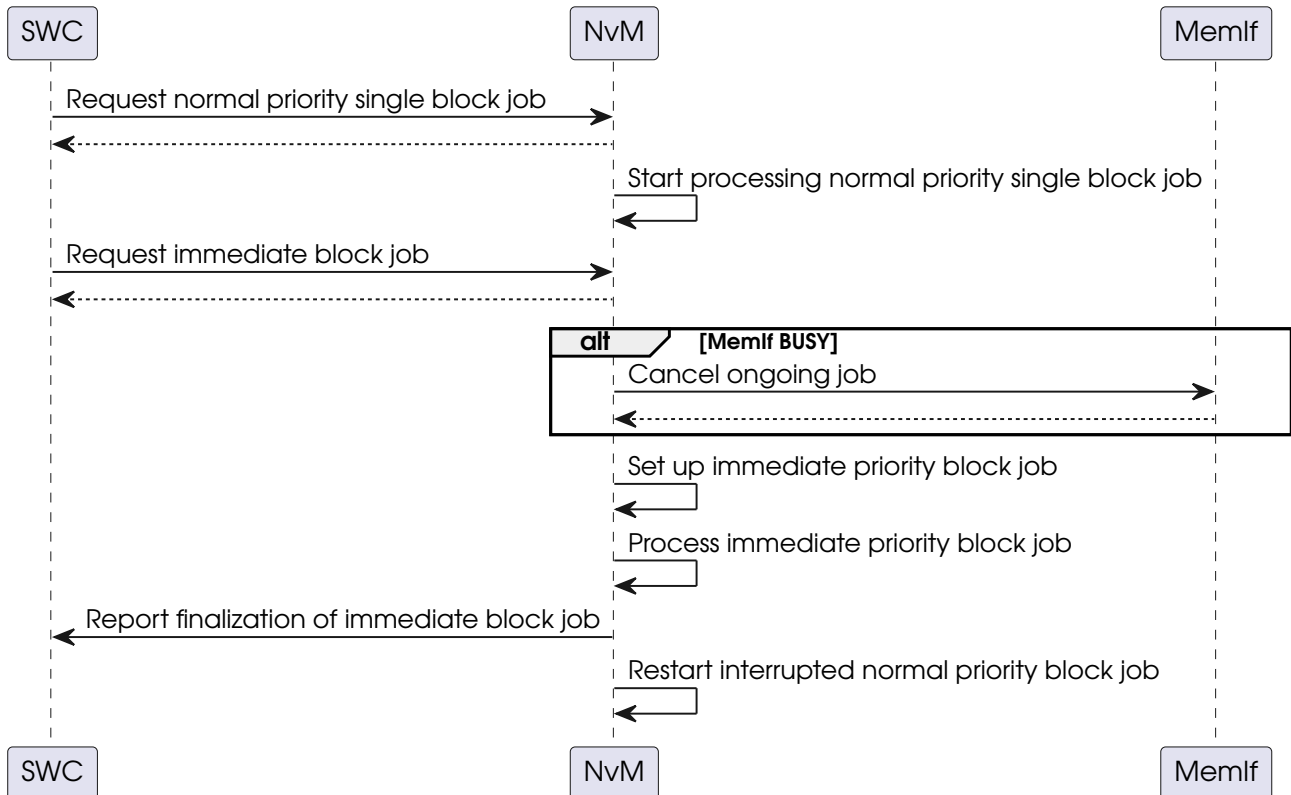


Figure 2.8 Interruption of a single block job by an immediate block job request



Caution

An already ongoing immediate block job is not interrupted by another immediate block job.

2.1.11.2 Multi Block Job Interruption

In case a multi block job is already ongoing when an immediate block job is requested, the ongoing multi block job will be interrupted and the immediate block job will be started right after the interruption. After the immediate block job is finished, the interrupted multi block job will be resumed starting from the block which was currently processed when the immediate block job was requested. The following diagram abstractly describes how the interruption of a multi block job is performed in case of an immediate block job request.

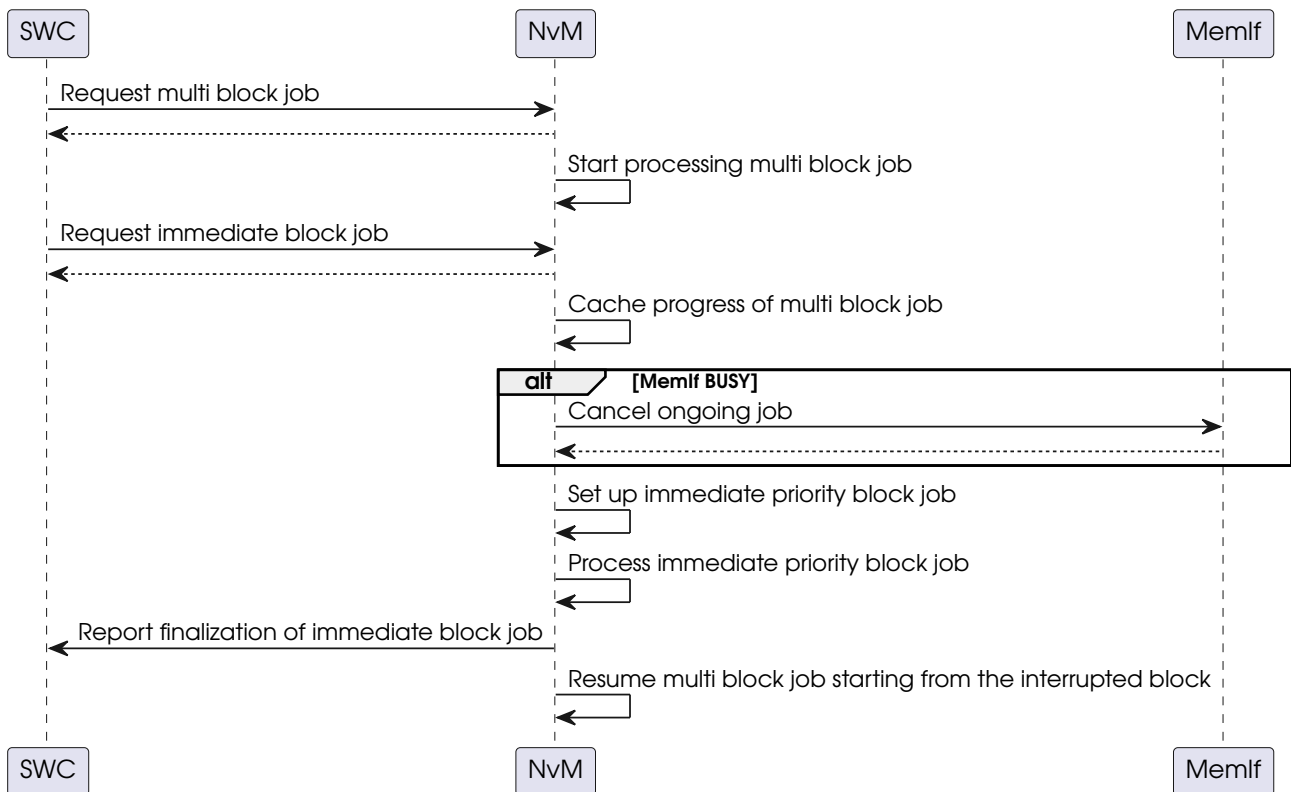


Figure 2.9 Interruption of a multi block job by an immediate block job request



Note

The interrupted individual block job of the multi block job processing is restarted from the beginning.



Caution

In case an immediate block is configured for ReadAll (resp. WriteAll), it might happen that a write request to this block is not accepted by the NvM in case a multi block job is already ongoing. Despite this fact, it is not recommended to use immediate block jobs while handling multi block jobs, as multi block jobs are very time sensitive and an immediate interruption will extend the overall processing time of these multi block jobs.

2.1.11.2.1 NvM_KillReadAll() and NvM_KillWriteAll() during immediate job processing

In case NvM_KillReadAll() or NvM_KillWriteAll() is called during immediate block job processing, the underlying MemIf is not canceled by the API call to avoid compromising the ongoing immediate block job. The following diagram abstractly describes the behavior of the NvM when NvM_KillReadAll() or NvM_KillWriteAll() is called during immediate block job processing.

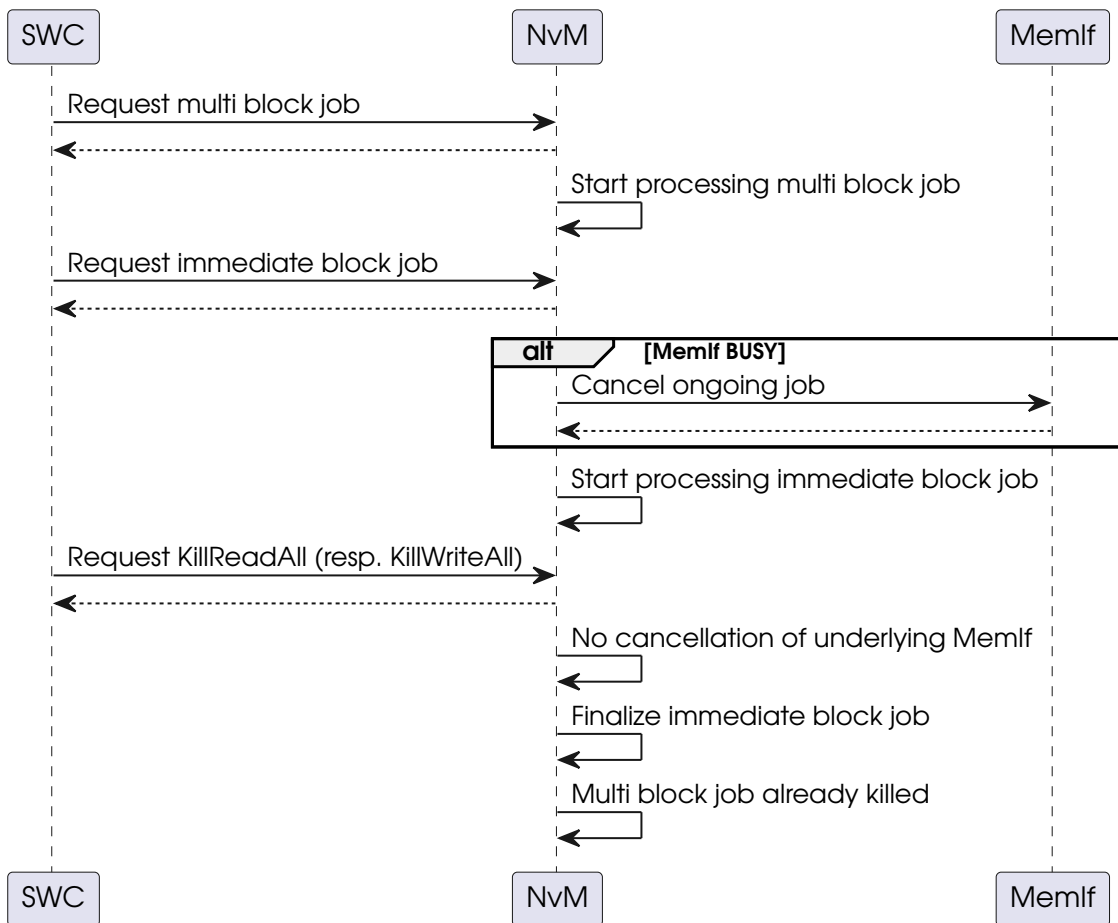


Figure 2.10 NvM_KillReadAll() or NvM_KillWriteAll() during immediate block job processing

2.1.11.2.2 NvM_CancelWriteAll() during immediate job processing

The service NvM_CancelWriteAll() behaves the same way in combination with immediate block job processing. The service terminates an ongoing WriteAll in a non-destructive way. The following diagram abstractly describes the behavior of the NvM when NvM_CancelWriteAll() is called during immediate block job processing.

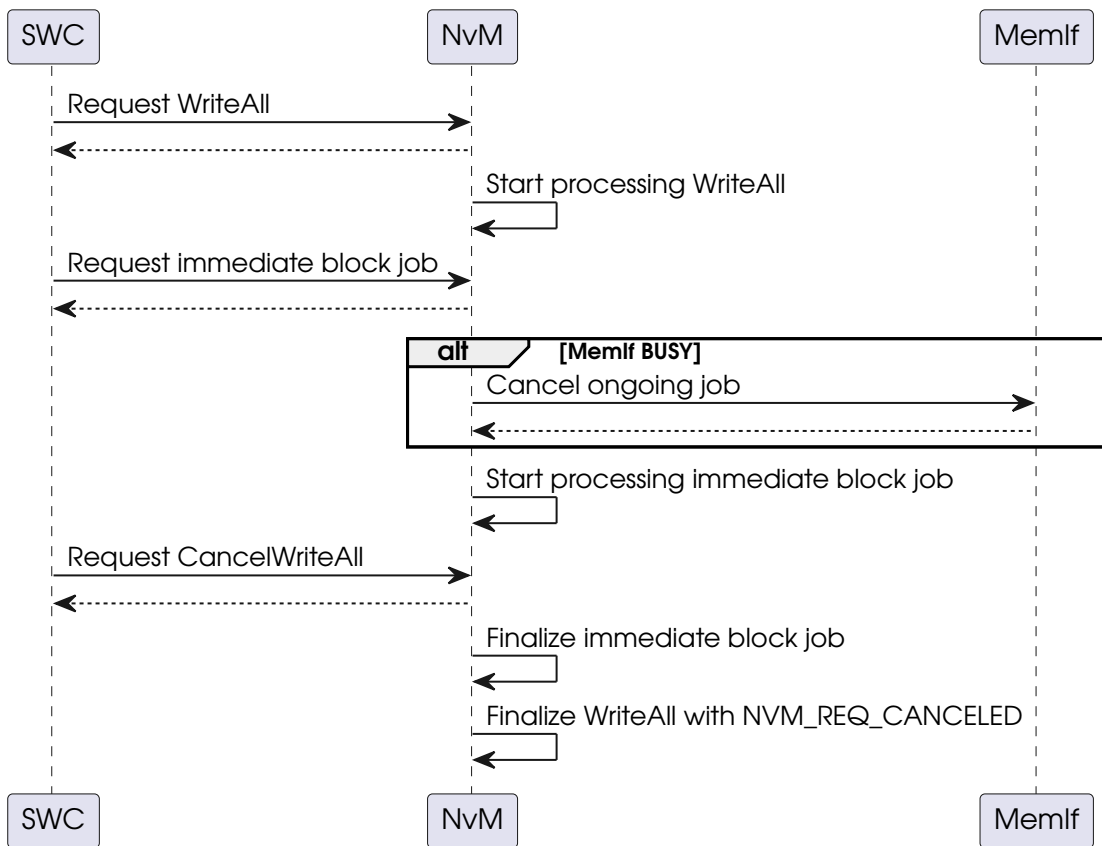


Figure 2.11 NvM_CancelWriteAll() during immediate block job processing



Caution

The block, which was processed during WriteAll before the immediate block job was requested, is not reprocessed again after the immediate block job is finished.

2.1.11.3 Specialties for Multi-partition Projects

The NvM also supports immediate block handling for multi-partition projects. In multi-partition case, there are some specialties to be considered for the immediate block job handling:

- > The cancel operation at the level of the underlying MemIf is only performed by the master partition.
- > In case a single block job is already finished at MemIf level and the block belongs to a partition different from the immediate block request, the job is still being finished and no progress is lost.
- > In case an immediate block job is requested at a specific satellite, this satellite (together with the master partition) has to be processed until the immediate block job is finished. The NvM will only continue processing other jobs after the immediate block job is finished.

**Expert Knowledge**

An immediate block job will be propagated to the master partition right after request, which will give the master partition the possibility to prepare for the immediate block job (e.g. potentially cancel an ongoing job at the level of the underlying MemIf). The master partition then waits for the immediate block job ready for processing. In the meantime, no other job is accepted by the master partition, avoiding the risk of accepting a job which potentially has to be canceled again.

2.1.11.4 General specialties

The following specialties are valid for both single and multi-partition projects:

- > It might happen that the progress of an interrupted block job is lost, because it is canceled at MemIf level and restarted after the immediate block job is finished.
- > An additional MemIf call might be performed in case an interrupted block job was already finished at MemIf level, because it is restarted after the immediate block job is finished.
- > Overall processing time of a single block job (resp. multi block job) is extended by the time needed for the immediate block job.

This feature was initially created considering AUTOSAR 23-11.

2.1.12 Erase Data

Erasing NV data is possible via the service API `NvM_EraseNvBlock()`

**Caution**

The API is only meant for blocks with configured IMMEDIATE priority

This feature was initially created considering AUTOSAR 23-11.

2.1.13 Data Integrity

The NvM provides multiple strategies to ensure the data integrity of a configured block, i.e., it can check if the block data read from NV-memory corresponds to the data that was previously written.

**Note**

Only a single data integrity mechanism can be configured for a block at a time. For different blocks different data integrity mechanisms can be configured. Additional features that require a dedicated strategy may not be available if a different data integrity mechanism is configured (e.g. MAC-based Data Integrity disables CRC-based features stated in Features only available for CRC mechanism).

2.1.13.1 Internal Data Integrity Buffer

If enabled, the parameter `NvM/NvMCommonVendorParams/NvMDataIntegrityIntBuffer` provides an internal buffer for the data integrity record for each block that has a data integrity mechanism enabled. This buffer is used to store the data integrity value after the data integrity record is calculated for a write job or the data integrity record is successfully validated during a read job.



Note

As the internal buffer is always available at the moment, the Ram Block Data Buffer (Implicit or Explicit Synchronization) must be only large enough to hold the payload data. So the parameter `NvM/NvMCommonVendorParams/NvMDataIntegrityIntBuffer` has no influence on the external and internal buffer handling for now.

2.1.13.2 Block Size Configuration

The parameter `NvM/NvMBlockDescriptor/NvMNvBlockLength` defines the length of a block.



Caution

The `NvM/NvMBlockDescriptor/NvMNvBlockLength` only represents the actual payload length, i.e. the length of the data integrity record is not included.

There is currently always an internal buffer, which ensures that payload and data integrity record can be handled correctly. The size of the internal buffer is the size of the biggest configured block plus the size of the configured data integrity record.

2.1.13.3 CRC-based Data Integrity

It is possible to configure automatic data integrity evaluation on a per block basis. To enable this, following parameters must be set:

- > `NvM/NvMBlockDescriptor/NvMBlockUseCrc=TRUE`
- > `NvM/NvMBlockDescriptor/NvMBlockCrcType=<value>`

Supported data integrity types are:

- > CRC16
- > CRC32

When enabling this feature, the configured CRC variant is automatically calculated and stored to the NV data when a block is written. Writing will always recreate the CRC and no mirror storing the previous value is available.

When reading the block, an additional CRC value is calculated and compared to the stored CRC value. This comparison is used to derive the data integrity of the block.

This feature was initially created considering AUTOSAR 23-11.



Expert Knowledge

CRC is not able to derive data integrity with 100% certainty. The calculation is highly depending on the size of both: the polynomial and the payload. In very rare cases it is possible that different data can lead to the same CRC value (=collision).

The probability of such a collision is extremely low, but not impossible.

2.1.13.3.1 Block ID Check

The Block ID Check feature ensures that the provided data by the underlying layers belongs to the requested block if enabled. The functionality can be switched on with the parameter `NvM/NvMCommon/NvMUseBlockIdCheck`.

To ensure that the data belongs to the currently processed block, the NvM takes the Block ID and the current Data index of the block into account when calculating the CRC. This means that the CRC value is calculated from the block ID (2 bytes), the data index (1 byte) and the actual payload of the currently processed block. If the feature is enabled, the CRC calculation needs an additional step as opposed to when it is not enabled.

The NvM is not able to distinguish between wrong payload CRC or CRC calculated for another block. In case the underlying layers provide data belonging to wrong block, the NvM behaves in same way as in case of payload CRC mismatch. In both cases, the error status is `NVM_REQ_NOT_OK` and can be retrieved via Monitor Job Status.



Caution

As the block ID check uses CRC to check whether the data belongs to the block, the feature only works for blocks with enabled CRC.



Note

This is a safety relevant feature, i.e. must be enabled to ensure safety of NV data.

2.1.13.4 MAC-based Data Integrity

For each block an optional MAC can be configured. For generating and verifying MACs, the CSM module is used. The generated MAC will be appended to the user data and both are stored in a single block in NV-memory.

For enabling MAC as a data integrity mechanism, the following parameters must be set:

- > `NvM/NvMBlockDescriptor/NvMCsmMacGenerationJobReference`
- > `NvM/NvMBlockDescriptor/NvMCsmMacVerificationJobReference`
- > `NvM/NvMBlockDescriptor/NvMCsmMacSize`

> NvM/NvMCommon/NvMCsmRetryCounter

Both `NvMCsmMacGenerationJobReference` and `NvMCsmMacVerificationJobReference` must reference the required CSM jobs to generate and verify the MAC, respectively. The referenced jobs must themselves reference the correct `CsmPrimitives` (`CsmMacGenerate` and `CsmMacVerify`) and the processing (i.e. `CsmMacGenerateProcessing` and `CsmMacVerifyProcessing`) must be set to `CSM_SYNCHRONOUS`. The parameter `NvMCsmMacSize` defines the actual size of the MAC that is generated from the block data. The parameter `NvMCsmRetryCounter` defines how often the MAC generation (resp. verification) is retried in case the Csm module is busy or the Csm internal queue is full.



Note

The MAC support is a non AUTOSAR feature.



Caution

When configuring MAC for a block, multiple features become unavailable that require CRC as data integrity mechanism (e.g. `NvMCalcRamBlockCrc`, `NvMUseBlockIdCheck` or `NvMBlockUseCRCCompMechanism`). Therefore the background CRC recalculation, skipping reading of valid data and skipping writing of unchanged data are also not available for blocks configured with MAC.

2.1.14 Recovery of Default Data

The module supports different triggers as well as mechanisms to recover default data. All options are exclusive to each other.

This feature was initially created considering AUTOSAR 23-11.

2.1.14.1 Mechanisms for Data Recovery

2.1.14.1.1 Recovery via ROM Block Data

By configuring the parameter `NvM/NvMBlockDescriptor/NvMRomBlockDataAddress=<symbol>`, the given symbol will be used as the start address for default data restoration. The NvM will simply copy the data from the symbol.



Caution

The NvM expects this data to have the same length as the block data.

2.1.14.1.2 Recovery via InitBlockCallback

By enabling `NvM/NvMBlockDescriptor/NvMUseInitCallback` the parameter `NvM/NvMBlockDescriptor/NvMInitBlockCallback=<symbol>` can be configured. Then the given symbol will be used as a callback routine. The implementation of this callback must take care of the correct restoration of default data for the related block. After the invocation the block is expected to be restored.



Expert Knowledge

The callback routine must follow the type definition of `NvM_InitBlockCbKPtrType`.

2.1.14.1.3 Recovery via InitBlockCallbackExtended

With `NvM/NvMBlockDescriptor/NvMUseInitCallback` disabled the parameter `NvM/NvMBlockDescriptor/NvMInitBlockCallbackExtended=<symbol>` can be configured. Then the given symbol will be used as a callback routine. The implementation of this callback must take care of the correct restoration of default data for the related block. After the invocation the block is expected to be restored. To learn about the reason for the extended callback see `Extended SingleBlock` and `InitBlockCallbacks`.



Expert Knowledge

The callback routine must follow the type definition of `NvM_ExtendedInitBlockCbKPtrType`.



Note

The extended init block callback can be used for every data synchronization mechanism (Data Synchronization), as well as for temporary buffers. Special case is the explicit synchronization mechanism, where after the extended init block callback is invoked, the `NvMReadRamBlockFromNvCallback` is called to ensure that the restored data is transferred to the application.

2.1.14.2 Triggers for Data Recovery

2.1.14.2.1 Explicit Recovery of Default Data

To explicitly recover default data of a certain block, the APIs `NvM_RestoreBlockDefaults()` / `NvM_RestorePRAMBlockDefaults()` can be used. Invoking this service will restore the default data depending on the configured recovery mechanism (see above).

**Note**

`NvM_RestoreBlockDefaults()` allows default data recovery also for a temporary buffer.

`NvM_RestorePRAMBlockDefaults` is the equivalent to `NvM_RestoreBlockDefaults(<BlockId>, NULL_PTR)`.

2.1.14.2 Implicit Recovery of Default Data

The module will automatically attempt to recover the default data, whenever a read to a block fails and a data recovery mechanism is configured.

Reasons for a failed read can be:

- > Lower layers did not accept the read request
- > Lower layer accepted the request and failed later
- > Read request succeeded but a data integrity error was detected

2.1.15 Write Protection

Enabled write protection of a block leads to:

- > Any Write/Erase/Invalidate request is rejected
- > NV Data is protected (during WriteAll)

To enable/disable the write protection either the `NvM_SetBlockProtection` API can be used or the block is write protected by enabling `NvM/NvMBlockDescriptor/NvMWriteBlockOnce`. See Write Block Once for more details.

The write protection of a block can also be initially enabled/disabled by setting the parameter `NvM/NvMBlockDescriptor/NvMBlockWriteProt` accordingly.

**Info**

Simultaneous activation of `NvM/NvMBlockDescriptor/NvMWriteBlockOnce` and `NvM/NvMBlockDescriptor/NvMBlockWriteProt` means that the block can never be written.

The following diagram shows a possible scenario for write protection when data must be written to an NV block after the read fails but the block is write protected (without write block once).

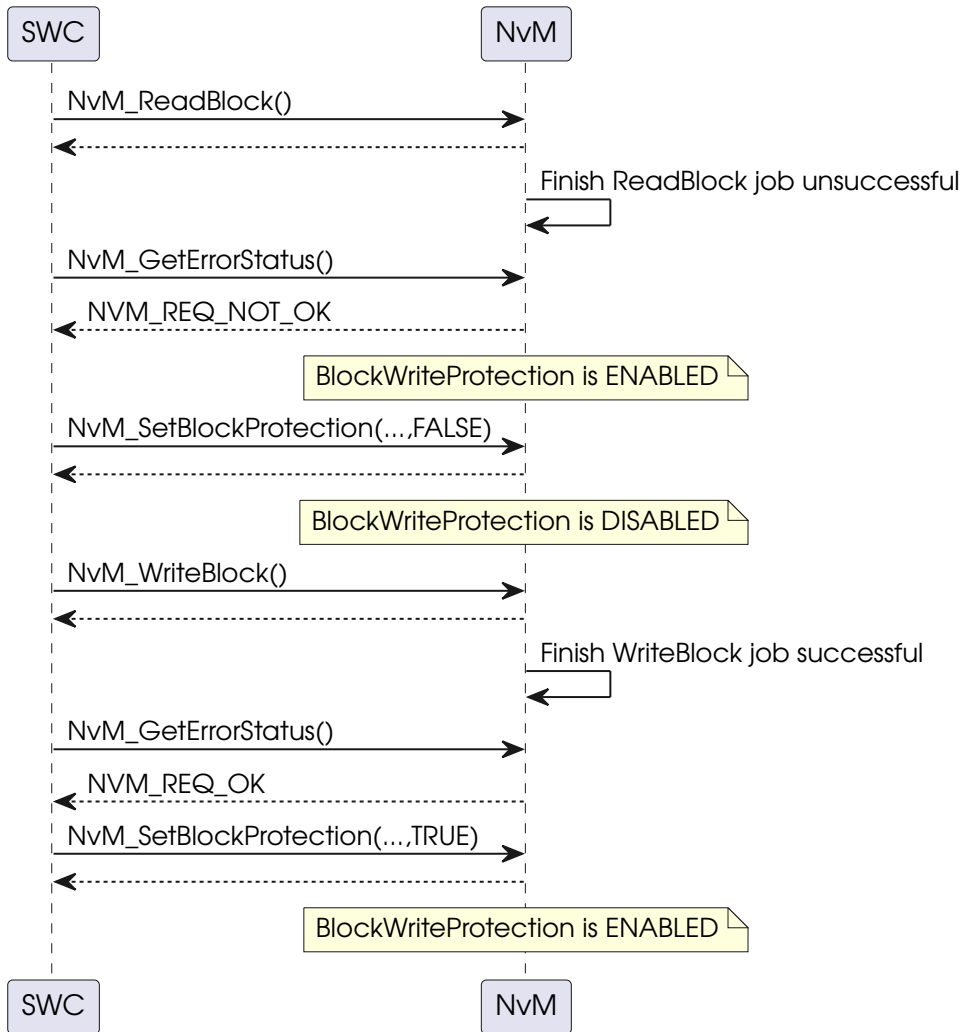


Figure 2.12 Example of Write Protection Scenario

This feature was initially created considering AUTOSAR 23-11.

2.1.15.1 Write Block Once

In some cases, it makes sense to save data only once in an NV block, i.e. overwriting of data shall be prohibited. To provide the possibility, the parameter `NvM/NvMBlockDescriptor/NvMWriteBlockOnce` has to be enabled.

The following scenarios leads to a write protection of a block if write block once is enabled:

- > Block is successfully read and data is valid
- > Block is successfully written



Info

If write block once is enabled, the write protection status of a block cannot be changed with `NvM_SetBlockProtection` API.

**Caution**

If write block once is enabled for a dataset block, only one dataset can be written. Requesting any other dataset, leads to a rejected Write/Erase/Invalidate request because the whole block is already write protected.

2.1.16 Write Retries

The NvM supports retries for write operations when failures are reported by the underlying layers. This feature can be enabled via the parameter `NvM/NvMCommon/NvMMaxNoOfWriteRetries` and it defines the maximum number of retries for a write operation. Setting the value to 0 disables the write retry feature.

**Info**

If Production Error Reporting is enabled, an error (`NVM_E_REQ_FAILED`) is reported to the DEM when the retry counter exceeds the maximum number of write retries.

**Note**

AUTOSAR defines a maximum number of write retries configurable per block, but the MICROSAR NvM currently supports a global configuration. This means that the configured value is applied to all blocks. This forces a write retry (if configured) for any write failure reported from the underlying layers.

This feature was initially created considering AUTOSAR 23-11.

2.2 Error Handling

2.2.1 Development Error Reporting

The module supports development error detection and reporting using the AUTOSAR Det. If enabled, these errors are reported using the services `Det_ReportError()` / `Det_ReportRuntimeError()`.

The reported service IDs and error IDs are contained and documented in the module header file.

**Info**

The component uses the same service for DET reporting from multiple partitions. MICROSAR Det supports this functionality by mapping a spinlock to its exclusive area. Please refer to the technical reference for further information.

This feature was initially created considering AUTOSAR 23-11.

2.2.2 Production Error Reporting

The component supports production error detection and reporting using the AUTOSAR Dem component.

If enabled, the following errors are reported using the service `Dem_SetEventStatus()`.

Error	Description
NVM_E_INTEGRITY_FAILED	Data integrity is not ensured. (MemIf provides MEMIF_BLOCK_INCONSISTENT result or CRC mismatch)
NVM_E_REQ_FAILED	The API request failed. (MemIf request rejects request or provides MEMIF_JOB_FAILED result, Explicit Synchronization mechanism fails)
NVM_E_LOSS_OF_REDUNDANCY	Redundant block has lost its redundancy.

Table 2.1 Errors reported to DEM

The NvM invokes the DEM API with given error code and the status `DEM_EVENT_STATUS_FAILED`.



Note

The legacy NvM has used different SID schemata and values. If the integration somehow interprets the reported IDs, please ensure that they are in sync with the defined SIDs in *NvM.h*

This feature was initially created considering AUTOSAR 23-11.

2.2.2.1 Multi-partition particularities

DEM error reporting for multi-partition projects has two different reporting scenarios:

- > The DEM error detected in the NV functionality are reported directly to the DEM
- > Errors detected in the application functionality are first forwarded to the master and later dispatched to the DEM within the master main function.



Note

The NvM Master reports DEM errors to any instance of the DEM (either Satellite or Master) depending on the configuration.

2.3 Concurrent access to NV data for DCM

NvM allows concurrent access to NV data alongside its application, particularly for diagnostic services. To support this use case, a separate block structure, known as the DCM block, is introduced. While the block management structure differs from the original block, all other properties, remain the same (e.g. Block length).



Caution

The WriteBlock/EraseNvBlock/InvalidateNvBlock requests for DCM blocks have to contain a valid temporary data buffer.

The following diagram shows how the concurrent access to the block can be handled.

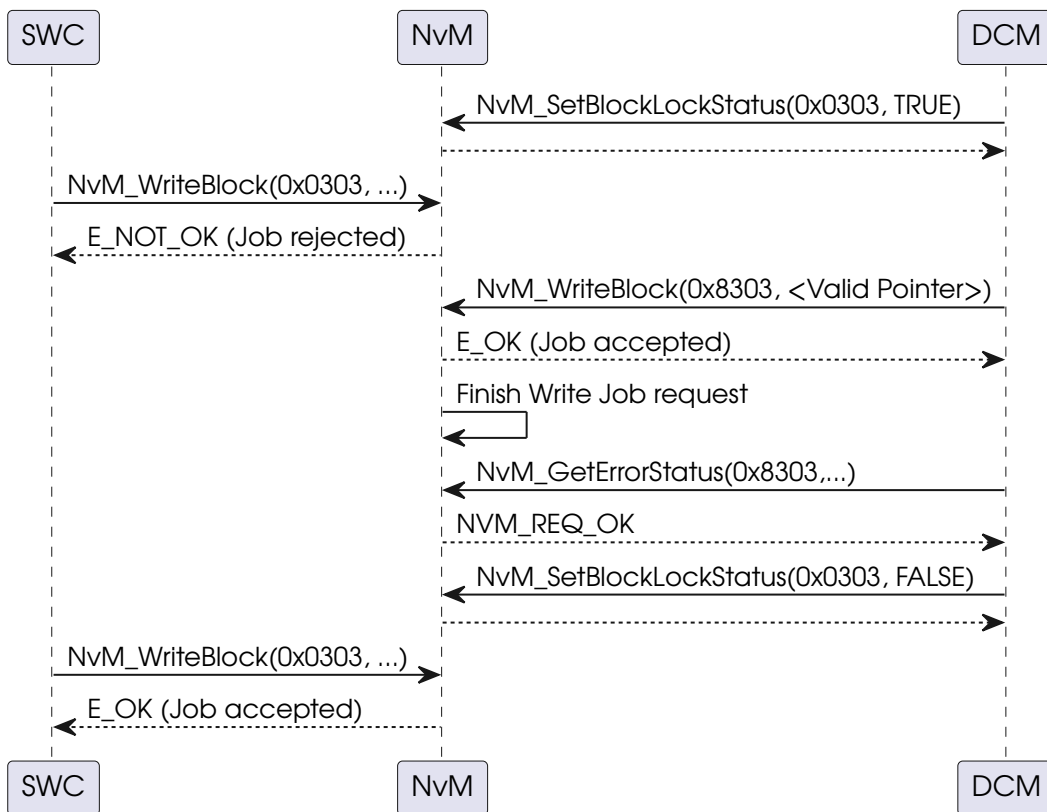


Figure 2.13 Example of Block Access



Note

The lock of the block only affects the original block (application block).



Note

The DCM block can be accessed via adding an offset (0x8000) to the origin block id. This offset is globally available via `NVM_DCM_BLOCK_OFFSET` define.



Caution

If dataset blocks must be handled via DCM access, make sure to set the data index of the DCM block to the desired value.

2.4 Compliance

These are the major unsupported features compared to AUTOSAR [1]

- > First Initialization (`NvM_FirstInitAll`)
- > Read Retry
- > Write Verification
- > Block Compression
- > Block Ciphering
- > Repair of redundant blocks during `NvM_WriteAll` if redundancy is lost
 - > Ignore write protection if necessary
- > Callback functions to be used by underlying memory abstraction
- > Make callbacks available via `NvM_Externals.h` if `NvMBlockHeaderInclude` is not configured
- > Reject Write/EraseNv/InvalidateNv Block request made prior to the first read if `NvM/NvMBlockDescriptor/NvMWriteBlockOnce` is enabled
- > Allow rewriting of `NvM/NvMBlockDescriptor/NvMWriteBlockOnce` enabled blocks when extended runtime is executed
- > Propagation of `NVM_REQ_INTEGRITY_FAILED` (if `MemIf` returns `MEMIF_BLOCK_INCONSISTENT`) and `NVM_REQ_NV_INVALIDATED`.

These are the major deviations compared to the used AUTOSAR version [1]

- > `NvM_InitBlockCallback` does not use the `NvM_InitBlockRequestType` as a parameter but uses void to be compatible with the MICROSAR Dem, as this component is implemented against the NvM with the AUTOSAR 4.X standard.
- > Additionally added `NvM/NvMBlockDescriptor/NvMInvokeCallbacksForReadAll=<value>` parameter for the assurance of correctly handling `NvMSingleBlockCallback` and `NvMInitBlockCallback` during `NvM_ReadAll()`.
- > Added `NvM_BackgroundCrcRecalcCallback` for the notification of a finished background crc recalculation of a block.
- > Instead of supporting the AUTOSAR defined `NvMMaxNumOfWriteRetries`, the MICROSAR NvM supports a more generic retry mechanism via the configuration parameter `NvM/NvMCommon/NvMMaxNoOfWriteRetries`.
- > The configuration of the parameter `NvM/NvMCommon/NvMEcucPartitionRef` is optional in order to support projects without OS applications.

There are no additional deviations of the supported features from AUTOSAR [1].

2.4.1 Compliance to previous NvM generation

This version of the NvM does not support the following features that were supported by previous NvM generation.

- > NvM_PreReadAll service
- > NvM_RepairRedundantBlocks service

Alternatives are planned for the above mentioned features in the new NvM generation.

2.4.2 Additions/Extensions

2.4.2.1 Support of AUTOSAR 4.4.0 NvMSingleBlockCallback, NvMMultiBlockCallback and NvMMultiBlockJobStatusInformation

With AUTOSAR version 4.4.0 the interfaces of `NvMSingleBlockCallback`, `NvMMultiBlockCallback` and `NvMMultiBlockJobStatusInformation` was changed in a way that backward compatibility was not given anymore.

To support legacy systems but also systems requiring the new interfaces the configuration parameter `NvM/NvMCommon/NvMUseAsr440CallbackInterface` was added. When enabling this parameter, the callbacks will be invoked with dedicated types (`NvM_BlockRequestType` / `NvM_MultiBlockRequestType`) instead of the former `ServiceId`.



Caution

Please ensure that the SWC components and BSW components use the correct NvM literals defined in `NvM_Types.h` when using the callbacks as the behavior has changed.

2.4.2.2 Support of NVM_REQ_RESTORED_x request result types

In AUTOSAR version 20-11 the name of the request result type `NVM_REQ_RESTORED_FROM_ROM` was changed to `NVM_REQ_RESTORED_DEFAULTS`. Due to backward compatibility reason, the NvM provides both request result type names via defines.

2.4.2.3 Extended SingleBlock and InitBlockCallbacks

In AUTOSAR the Single and InitBlockCallbacks are very limited in the data being passed to the callback. As no information about the Nv-block is passed it is nearly impossible to re-use a callback function for multiple Nv-blocks without the usage of a wrapper or similar mapping layer. To circumvent this limitation extended versions of these callbacks are offered. In both cases the extended version additionally supplies the `BlockId`. In case of the `ExtendedSingleBlockCallback` the address of the RAM mirror and the `BlockLength` are also provided.

3 Integration

NvMGen2 supports native multi-partition handling of memory blocks. Functionally, there is no difference between single/multi-partition projects. From an integrational stand point there are major differences and additional actions to be considered.

The following chapters concentrate on single partition projects **only**. A dedicated chapter is introduced at the end of this chapter for multi-partition projects (Multi-Partition Projects).

3.1 Embedded Implementation

The component contains the following source code files. Some files are generated by the configuration tool DaVinci Configurator Classic.

File Name	Description	Integration Tasks
NvM.c	Source file of the component.	-
NvM.h	Header file that defines services used by other components. Only this file shall be included by the application/user.	-
NvM_Types.h	Header file that defines types used in services of the component.	-
NvM_Cfg.c	This is a generated source file containing the NvM configuration, e.g. the configured blocks.	-
NvM_Cfg.h	Generated header file that contains pre-compile switches which configure the component features. It also declares pre-compile configurable data. The file will be included by NvM.h and therefore provided to the application/ user.	-
NvM_Partition-Functions.h	This is a generated header file containing satellite main functions wrapper declarations. Note: Only available in multipartition usage scenario.	-
NvM_Partition-Functions.c	This is a generated source file containing satellite main functions wrapper definitions. Note: Only available in multipartition usage scenario.	-
NvM_Swc_Types.h	This is a generated header file includes all relevant RTE type headers. Relevant for providing callback declarations, etc. to the NvM.	-
NvM_MemMap.h	This is a generated header file containing memory mapping sections.	-

Table 3.1 Source Code Files

3.2 Initialization

Before calling any functionality, the module has to be initialized by calling NvM_Init(). As the NvM does not call service functions of other modules during initialization, the NvM can be initialized before any other adjacent module is initialized.

**Caution**

FEE/EA must be initialized first in order to be able to use functionality of the NvM, such as `NvM_ReadAll()`.

3.3 Main Functions

The module has one main function:

> `NvM_MainFunction()`

This main function has to be called on task level by the BSW Scheduler with a

- > fixed cycle time, or
- > within a background task.

This behaviour can be changed via `NvM/NvMCommon/NvMMainFunctionTriggering`.

**Caution**

The option `FixedCycleAndBackground` is currently not supported.

The cycle time `NvM/NvMCommon/NvMMainFunctionPeriod` can be configured in the configuration tool.

3.4 Synchronization

3.4.1 Critical Sections

When multiple processes or threads have the potential to access and modify internal data structures simultaneously, it becomes crucial to ensure that only one process or thread can enter the critical section at any given time. This helps prevent scenarios where data integrity might be compromised due to conflicting modifications

To protect its internal data structures against unwanted modification, the NvM uses following critical section for blocking concurrent access:

> `NvM_NVM_EXCLUSIVE_AREA_0`

This critical section covers trivial code section with trivial sub-functions and has a short run-time.

3.5 Memory Sections

The component does not require any special integration tasks for its memory sections.

**Caution**

The user shall not map variables to cached RAM. This is because not only the CPU, but also the memory hardware may access the RAM directly. If the CPU accesses cached RAM while the memory hardware accesses the same RAM region concurrently, it may lead to data inconsistencies.

3.6 Callouts

The NvM does not have any callout functions.

3.7 Error Reporting Det

The component has these properties used for error reporting to the Det component.

> NVM_MODULE_ID = 20

> NVM_VENDOR_ID = 30

3.8 Block Length Features

3.8.1 Automatic Block Length

The Automatic Block Length feature provides the possibility to determine the size of a permanent RAM block during compile time for *NvMRamBlockDataAddress* configured blocks.

The feature changes the configured block length to the determined block length which also alters the meaning of the block length from actual length to maximum length. The actual block length is set by the *sizeof()* operator via the permanent RAM within the generated structures. The configured length is used by underlying modules to initialize their structures, therefore it must not be less than the actual length. To check the configured length to be valid, Block Length Check shall be enabled.

**Caution**

After the system is running and the actual block lengths are known, the configuration shall be adjusted to the actual length. Since the configured lengths are used by underlying modules, there might be a lot of unused space in Flash or EEPROM.

3.8.2 Block Length Check

The module provides a block length check for permanent RAM or ROM blocks to ensure that the configured block length and the actual length of the RAM or ROM block match.

The following matching mechanisms are available:

Mechanism	Description
Automatic Block Length enabled	Size of actual RAM or ROM block length must not exceed the configured block length
Strict	Configured block length has to match the size of the actual permanent RAM or ROM block length exactly
Non-Strict	Configured block length must not exceed the size of the permanent RAM or ROM block length

Table 3.2 Matching Mechanisms



Basic Knowledge

The module uses bitfields to check the block length during compile time. The value of the bitfields are initialized with positive length. If the value is initialized with negative length, which means that the block length check has failed, a compiler error occurs and the corresponding line is marked.

Each block length check shows the following information:

- > Block Name
- > RAM/ROM symbol
- > Failure, depending on the matching mechanism



Note

This feature is not specified by AUTOSAR. It is only provided by MICROSAR to help integrating the NvM. The Block Length check uses the *sizeof()* operator, therefore the sizes are depended of the compiler version, the padding and alignment configuration.



Caution

As stated in Block Size Configuration the *NvMNvBlockLength* represents the actual payload length. Therefore, the Block Length Check only takes the actual payload length into account resulting in the configured block length and the permanent RAM must not contain the space for holding a Data Integrity Record.

3.8.3 Block Length Features for Explicit Synchronization Blocks

The block length features are only working with a given symbol in e.g. *NvMRamBlockDataAddress*. In some use cases it makes sense to support this features for explicit synchronization blocks.

AUTOSAR does not allow the combination of the parameters *NvMRamBlockDataAddress* and *NvMBlockUseSyncMechanism*. If enabling the *NvMDataSyncParameterCombination* parameter, the combination of both above mentioned parameters is possible.

Summary:

- > Explicit Synchronization is configured as the data synchronization mechanism
- > The symbol of the used data buffer must be configured in the *NvMRamBlockDataAddress* parameter
- > The block length check and automatic block length is then carried out using the specified data buffer

3.9 Multi-Partition Projects

The component supports the management of NV blocks distributed across multiple cores and/or partitions. Following diagram shows a simplified example overview of the architecture of the module when used in a multi-partition scenario.

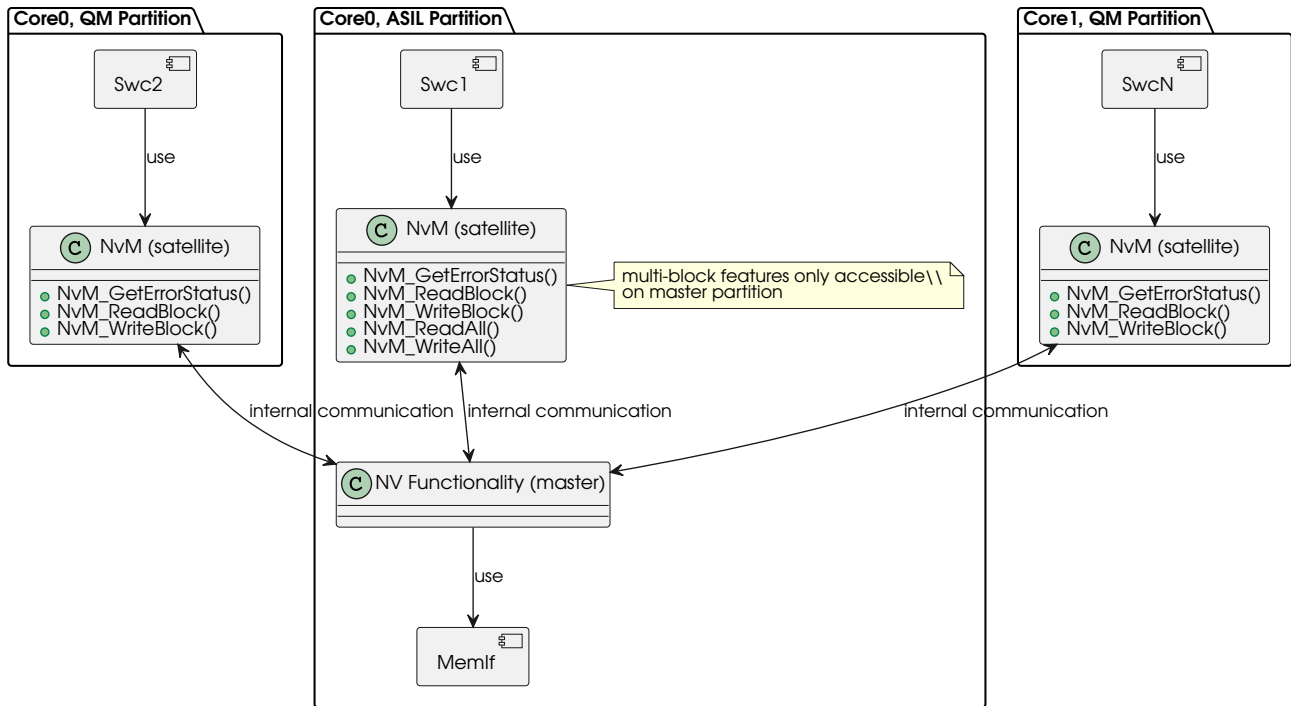


Figure 3.1 Simplified multi-partition Architecture



Info

The module supports a Mixed-ASIL setup which means that the partitions can have different ASIL level.
The master functionality can also be located in a QM partition.

3.9.1 Multi-Partition communication

The multi-partition communication logic assumes that accessing a *uint8* value is atomic. Consequently, it does not require any additional locking mechanisms.

3.9.2 Multi-Partition Projects and Out-Of-Order Instruction Execution



Caution

The implementation of multi-partition communication on BSW level relies on exclusion of manufacturer specific optimizations that lead to out-of-order instruction execution in the processor.

3.9.3 Configurational Particularities

Following parameters must be configured for multi-partition projects:

Configuration Parameter	Description
NvM/NvMCommon/ NvMEcucPartitionRef	Must contain all relevant ECUC partition references
NvM/NvMCommon/ NvMMasterEcucPartitionRef	Identifies the master partition <i>note: only required for multi-partition projects</i>
NvM/NvMBlockDescriptor/ NvMBlockEcucPartitionRef	Each block must be assigned to a EcucPartition <i>note: only required for multi-partition projects</i>

Table 3.3 Required parameters for multi-partition projects

3.9.4 Integrational Particularities

3.9.4.1 Partitioning

The module runs the same code for each partition. Runtime variables and management data are therefore generated multiple times.

For the public API as well as for the main functions, the module uses the OS routine `GetApplicationID()` to determine the call context and load the corresponding set of internal management data and runtime variables.

3.9.4.1.1 Main Functions

Instead of a single `NvM_MainFunction()`, a multi-partition project has multiple instances of main functions:

- > `NvM_MainFunctionMaster()`
- > `NvM_MainFunction_<OsApplicationName>` (satellite main function for each partition)

During the execution of a main function or API, the module uses the OS routine `GetApplicationID()` to determine the correct call context (e.g. to load the correct set of internal management information).



Caution

The integrator is responsible to ensure that the execution of main functions and API invocations lead to the correct result of `GetApplicationID()`. The module has implemented DET checks for single block services that can detect unallowed access to a block from a foreign partition. Main functions must be mapped to the corresponding tasks, no additional check is performed for them.

3.9.4.1.2 Memory Sections

In multi-partition projects, the module provides memory sections with the following pattern for each partition:

```
NVM_<START/STOP>_SEC_<OsApplicationName>_<SectionType>
```

These sections can be used to provide security of unallowed access to safe partitions via MMU/MPU or similar.

3.9.4.2 Startup / Shutdown of MP projects (ReadAll, WriteAll)

As indicated above, the integration of a multi-partition project is not trivial. This is especially true for the multiblock jobs `NvM_ReadAll()` and `NvM_WriteAll`.

It requires thorough integration to get a functional and performant solution. We provide a separate document concentrating solely on this purpose, see [2].

3.10 Projects without OS Application

The component also supports projects that do not contain the OS component. In this use case, the public API including the main function is equal to the one for single partition projects with OS.

3.10.1 Configurational Particularities

To configure the use case, leave the parameter `NvM/NvMCommon/NvMEcucPartitionRef` empty.

4 API Description

4.1 Type Definitions

The types defined by the component are declared in `NvM_Types.h`.

4.2 Provided Functions

The component provides the following functions.

Service functions are implemented by the component and are typically called by upper-layer components. They are declared in `*.h`.

4.2.1 NvM_CancelJobs

Prototype	
Std_ReturnType NvM_CancelJobs (NvM_BlockIdType BlockId)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
Return Codes	
E_OK: The job was successfully removed from queue. E_NOT_OK: The job could not be found in the queue.	
Functional Description	
Service to cancel all jobs pending for a NV block.	
Particularities	
Module is initialized.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.1 NvM_CancelJobs

4.2.2 NvM_CancelWriteAll

Prototype
void NvM_CancelWriteAll ()
Functional Description
Cancel pending write all job Request to cancel a pending WriteAll request.
Options
callcontext: TASK reentrant: FALSE synchronous: FALSE

Table 4.2 NvM_CancelWriteAll

4.2.3 NvM_EraseNvBlock

Prototype	
Std_ReturnType NvM_EraseNvBlock (NvM_BlockIdType BlockId)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to erase a NV block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.3 NvM_EraseNvBlock

4.2.4 NvM_GetActiveMultiBlockApplicationId

Prototype	
Std_ReturnType NvM_GetActiveMultiBlockApplicationId(ApplicationType* ApplicationId)	
Parameters	
ApplicationId [out]	Current Application Id.
Return Codes	
E_OK: Request has been accepted. E_NOT_OK: > Request has not been accepted or no active partition available due to multiblock job not active. > ValidateAll request is ongoing, because all partitions are processed in parallel instead of sequentially as for the other multiblock jobs.	
Functional Description	
Get the current application id of the processing multiblock job.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.4 NvM_GetActiveMultiBlockApplicationId

4.2.5 NvM_GetDataIndex

Prototype	
Std_ReturnType NvM_GetDataIndex (NvM_BlockIdType BlockId, uint8 * DataIndexPtr)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
DataIndexPtr [out]	Pointer to where to store the current dataset index (0..255)
Return Codes	
E_OK: The index position has been retrieved successfully. E_NOT_OK: An error occurred.	
Functional Description	
Service for getting the currently set DataIndex of a dataset NVRAM block.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.5 NvM_GetDataIndex

4.2.6 NvM_GetErrorStatus

Prototype	
Std_ReturnType NvM_GetErrorStatus (NvM_BlockIdType BlockId, NvM_RequestResultType * RequestResultPtr)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
RequestResultPtr [out]	Pointer where the result is stored. Parameter must not be NULL.
Return Codes	
E_OK: The block dependent error/status information was read successfully. E_NOT_OK: An error occurred.	
Functional Description	
Service to read the block dependent error/status information.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.6 NvM_GetErrorStatus

4.2.7 NvM_GetVersionInfo

Prototype	
void NvM_GetVersionInfo (Std_VersionInfoType * Versioninfo)	
Parameters	
Versioninfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Functional Description	
Returns the version information NvM_GetVersionInfo() returns version information, vendor ID and AUTOSAR module ID of the component.	
Options	
callcontext: TASK ISR2 reentrant: TRUE synchronous: TRUE	

Table 4.7 NvM_GetVersionInfo

4.2.8 NvM_Init

Prototype
void NvM_Init ()
Functional Description
Initialization function Service to initialize the NvM.
Particularities
> Interrupts are disabled. > Module is uninitialized.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.8 NvM_Init

4.2.9 NvM_InvalidateNvBlock

Prototype	
Std_ReturnType NvM_InvalidateNvBlock (NvM_BlockIdType BlockId)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to invalidate a NV block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.9 NvM_InvalidateNvBlock

4.2.10 NvM_KillReadAll

Prototype
void NvM_KillReadAll ()
Functional Description
Kills an ongoing ReadAll job. The function signals the request to kill an ongoing ReadAll and returns - the actual killing will be done by NvM processing.
Options
callcontext: TASK reentrant: FALSE synchronous: FALSE

Table 4.10 NvM_KillReadAll

4.2.11 NvM_KillWriteAll

Prototype
void NvM_KillWriteAll ()
Functional Description
Kills pending write all job. Request to kill a pending WriteAll request, i.e. To cancel it destructively (in contrast to NvM_CancelWriteAll). It shall only called by EcuM (or comparable SW)
Options
callcontext: TASK reentrant: FALSE synchronous: FALSE

Table 4.11 NvM_KillWriteAll

4.2.12 NvM_MainFunction

Prototype
void NvM_MainFunction ()
Functional Description
Scheduled function of the NvM. It processes the requested NvM jobs.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.12 NvM_MainFunction

4.2.13 NvM_ReadAll

Prototype
void NvM_ReadAll ()
Functional Description
Initiates a multi block read request.
Options
callcontext: TASK reentrant: FALSE synchronous: FALSE

Table 4.13 NvM_ReadAll

4.2.14 NvM_ValidateAll

Prototype
void NvM_ValidateAll ()
Functional Description
Initiates a multi block validation request.
Options
callcontext: TASK reentrant: FALSE synchronous: FALSE

Table 4.14 NvM_ValidateAll

4.2.15 NvM_ReadBlock

Prototype	
Std_ReturnType NvM_ReadBlock (NvM_BlockIdType BlockId, void * NvM_DstPtr)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
NvM_DstPtr [out]	Pointer to the RAM data block.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to copy the data of the NV block to its corresponding RAM block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.15 NvM_ReadBlock

4.2.16 NvM_ReadPRAMBlock

Prototype	
Std_ReturnType NvM_ReadPRAMBlock (NvM_BlockIdType BlockId)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to copy the data of the NV block to its corresponding PRAM block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.16 NvM_ReadPRAMBlock

4.2.17 NvM_RestoreBlockDefaults

Prototype	
Std_ReturnType NvM_RestoreBlockDefaults (NvM_BlockIdType BlockId, void * NvM_DstPtr)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
NvM_DstPtr [out]	Pointer to the RAM data block.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to copy the data of the RAM block to its corresponding NV block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.17 NvM_RestoreBlockDefaults

4.2.18 NvM_RestorePRAMBlockDefaults

Prototype	
Std_ReturnType NvM_RestorePRAMBlockDefaults (NvM_BlockIdType BlockId)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to copy the data of the PRAM block to its corresponding NV block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.18 NvM_RestorePRAMBlockDefaults

4.2.19 NvM_SetBlockLockStatus

Prototype	
void NvM_SetBlockLockStatus (NvM_BlockIdType BlockId, boolean BlockLocked)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
BlockLocked [in]	TRUE: Block Locked for application FALSE: Block Unlocked for application
Functional Description	
Service for setting the lock status of a permanent RAM block or of the explicit synchronization of a NVRAM block.	
Particularities	
Module is initialized.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.19 NvM_SetBlockLockStatus

4.2.20 NvM_SetBlockProtection

Prototype	
Std_ReturnType NvM_SetBlockProtection (NvM_BlockIdType BlockId, boolean ProtectionEnabled)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
ProtectionEnabled [in]	TRUE: Write protection shall be enabled FALSE: Write protection shall be disabled
Return Codes	
E_OK: The block was enabled/disabled as requested E_NOT_OK: An error occurred.	
Functional Description	
Service for setting/resetting the write protection for a NV block.	
Particularities	
Module is initialized.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.20 NvM_SetBlockProtection

4.2.21 NvM_SetDataIndex

Prototype	
Std_ReturnType NvM_SetDataIndex (NvM_BlockIdType BlockId, uint8 DataIndex)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
DataIndex [in]	Index position of a NV/ROM block.
Return Codes	
E_OK: The index position was set successfully. E_NOT_OK: An error occurred.	
Functional Description	
Service for setting the DataIndex of a dataset NVRAM block.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.21 NvM_SetDataIndex

4.2.22 NvM_SetRamBlockStatus

Prototype	
Std_ReturnType NvM_SetRamBlockStatus (NvM_BlockIdType BlockId, boolean BlockChanged)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
BlockChanged [in]	TRUE: Validated the RAM block and mark block as changed. FALSE: Invalidate the RAM block and mark block as unchanged.
Return Codes	
E_OK: The status of the permanent RAM block or the explicit synchronization was changed as requested. E_NOT_OK: An error occurred.	
Functional Description	
Service for setting the RAM block status of a permanent RAM block or the status of the explicit synchronization of a NVRAM block.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.22 NvM_SetRamBlockStatus

4.2.23 NvM_WriteAll

Prototype
void NvM_WriteAll ()
Functional Description
Initiates a multi block write request.
Options
callcontext: TASK reentrant: FALSE synchronous: FALSE

Table 4.23 NvM_WriteAll

4.2.24 NvM_WriteBlock

Prototype	
Std_ReturnType NvM_WriteBlock (NvM_BlockIdType BlockId, const void * SrcPtr)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
SrcPtr [in]	Pointer to the RAM data block.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to copy the data of the RAM block to its corresponding NV block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.24 NvM_WriteBlock

4.2.25 NvM_WritePRAMBlock

Prototype	
Std_ReturnType NvM_WritePRAMBlock (NvM_BlockIdType BlockId)	
Parameters	
BlockId [in]	Unique identifier for one NvRAM block descriptor.
Return Codes	
E_OK: request has been accepted. E_NOT_OK: request has not been accepted.	
Functional Description	
Service to copy the data of the PRAM block to its corresponding NV block.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.25 NvM_WritePRAMBlock

4.3 Required Functions

The component requires the following functions from other components. Refer to the respective documentation for details.

Component	API							
MemIf	MemIf_Read	MemIf_Write	MemIf_EraseIm-mediate-Block	MemIf_Invali-date-Block	MemIf_Cancel	MemIf_SetMode	MemIf_GetJo-bResult	MemIf_GetSta-tus
Crc	Crc_Cal-culate-CRC16	Crc_CalculateCRC32						
Csm	Csm_Mac-Gener-ate	Csm_MacVerify						
Os	GetApplicationID							

continues on next page

Table 4.26 – continued from previous page

Component	API						
SchM	SchM_Enter_NvM_NVM_EXCLUSIVE_AREA_0	SchM_Exit_NvM_NVM_EXCLUSIVE_AREA_0					
VStdLib	VStdLib_MemCpy						

Table 4.26 Required Functions

4.4 Service Ports

This component provides the following service ports:

4.4.1 Client Server Interface

A client server interface is related to a Provide Port at the server side and a Require Port at client side.

4.4.1.1 Provided Ports

4.4.1.1.1 NvM_Service

Service Interface to Access NVRAM Blocks.

This Interface actually describes a family of Port Interfaces, as the availability of some operations depend on block configuration, as well as on NVM's global configuration .

Each distinct configuration results in a distinct Port Interface. These distinct configurations result from the possible combinations of Dataset (yes/no) and ROM Defaults (yes/no):

1. Dataset Blocks without ROM Defaults
2. Dataset Blocks with ROM Defaults
3. non-Dataset Blocks (native, redundant) without ROM Defaults
4. non-Dataset Blocks with ROM Defaults

Operation	API Function	Port Defined Argument Values
EraseBlock	NvM_EraseBlock	NvM_BlockIdType 1..n
GetDataIndex	NvM_GetDataIndex	NvM_BlockIdType 1..n
GetErrorStatus	NvM_GetErrorStatus	NvM_BlockIdType 1..n
InvalidateNvBlock	NvM_InvalidateNvBlock	NvM_BlockIdType 1..n

continues on next page

Table 4.27 – continued from previous page

Operation	API Function	Port Defined Argument Values
GetErrorStatus	NvM_GetErrorStatus	NvM_RequestResultType
ReadBlock	NvM_ReadBlock	NvM_BlockIdType 1..n
RestoreBlockDefaults	NvM_RestoreBlockDefaults	NvM_BlockIdType 1..n
SetDataIndex	NvM_SetDataIndex	NvM_BlockIdType 1..n
SetRamBlockStatus	NvM_SetRamBlockStatus	NvM_BlockIdType 1..n
WriteBlock	NvM_WriteBlock	NvM_BlockIdType 1..n

Table 4.27 NvM_Service

4.4.1.1.2 NvMAdmin

Interface to administer a specific NVM Block.

There will be one Port-Prototype per configured block.

Operation	API Function	Port Defined Argument Values
SetBlockProtection	NvM_SetBlockProtection	NvM_BlockIdType 1..n

Table 4.28 NvMAdmin

4.4.1.2 Required Ports

4.4.1.2.1 NvM_NotifyInitBlock

(Optional) PortInterface allowing a client SWC to dynamically provide Default Data for an NVRAM Block.

Operation	API Function	Port Defined Argument Values
InitBlock	-	-

Table 4.29 NvM_NotifyInitBlock

4.4.1.2.2 NvM_NotifyJobFinished

(Optional) Interface to notify a client SWC about completion of an asynchronous operation on associated NVRAM Block

Operation	API Function	Port Defined Argument Values
JobFinished	-	NvM_BlockRequestType NvM_RequestResultType

Table 4.30 NvM_NotifyJobFinished

5 Configuration

The functionalities of the component are configured with DaVinci Configurator Classic.

For details on configuration options refer to the component-specific descriptions that are shown in DaVinci Configurator Classic.

5.1 Configuration Variants

This component supports these configuration variants:

> VARIANT-PRE-COMPILE

The configuration classes of the parameters depend on the supported configuration variants. For their definitions please see the component-specific BSWMD ARXML file.

6 Cybersecurity

This chapter describes relevant information for a secure integration and configuration, so-called Cybersecurity Manual Items (CMI), of this component to fulfil identified Technical Cybersecurity Requirements (TCR). Additionally, functional cybersecurity dependencies to other components are described.

The NvM does not define any Cybersecurity Manual Items.

TCRs that are allocated to the NvM cannot be fulfilled as the corresponding features are not supported, yet.

7 Glossary and Abbreviations

API

Application Programming Interface

AUTOSAR

Automotive Open System Architecture

BSW

Basis Software

DEM

Diagnostic Event Manager

DET

Development Error Tracer

ECU

Electronic Control Unit

ISR

Interrupt Service Routine

MICROSAR

Microcontroller Open System Architecture (the Vector AUTOSAR solution)

RTE

Runtime Environment

SWC

Software Component

SWS

Software Specification

8 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com